INFRAESTRUTURA / TRANSPORTE DE E-MAIL

MANUAL DE INVESTIGAÇÃO DE ENTREGA DE E-MAILS

SMTP, MX, filas e logs: da submissão à transferência, às novas tentativas, ao greylisting e ao rastreamento no destinatário.

CONSULTA OPERACIONAL

Roteiros, comandos, evidências e cinco incidentes comentados

PRÁTICA REPRODUZÍVEL

Simulador SMTP com STARTTLS e 22 testes · Postfix real gerando logs, fila e DSN

Edição de consulta • Outubro de 2026 • Base: RFC 5321, 5322, 3207, 3463, 3464, 6647, documentação do Postfix e diretrizes do Gmail

Exemplos com domínios .example e .test e endereços reservados para documentação. Nenhuma mensagem foi enviada a destinatários externos.

EDIÇÃO 2 / 29 DE OUTUBRO DE 2026 / 48 PÁGINAS

Como utilizar este manual

Este manual liga os estudos de DNS e SPF à investigação do transporte de mensagens. O objetivo é localizar a etapa que falhou e demonstrar até onde a mensagem foi aceita, sem confundir transferência SMTP com entrega na caixa postal.

Três rotas de leitura

Objetivo	Sequência sugerida
Aprender o percurso	Fundamentos (capítulos 1 a 4), sessão SMTP (5 a 12), filas e evidências (13 a 20).
Atender um incidente	Método (capítulo 21), o caso mais parecido (22 a 26), relatório (35) e a referência rápida do Apêndice C, que aponta o capítulo de cada sintoma.
Praticar localmente	Laboratório (capítulos 27 a 34), exercícios (Apêndice A) e gabarito (Apêndice B).

Convenções

- Comandos usam shell Linux, salvo indicação; nomes de serviço e arquivos variam por distribuição e implantação. Quando o pacote difere entre famílias, isso é dito com `DEB` (Debian/Ubuntu) e `RPM` (RHEL e derivados).
- Saídas marcadas `EXECUTADO` vieram de execução real nesta edição, no simulador ou no Postfix 3.8.6 de laboratório. As demais transcrições são exemplos construídos e estão identificadas como tal.
- Referências numeradas entre colchetes estão no Apêndice D.

Pacote de laboratório

O PDF incorpora `laboratorio_smtp.zip` com o simulador, os testes, a preparação de TLS e a configuração de Postfix usada no capítulo 31. Extraia pelo painel de anexos do leitor de PDF ou, no Linux, com `pdfdetach -saveall`. O pacote não contém chave privada.

O que mudou na Edição 2

- Novos capítulos: **15** (greylisting), **16** (exigências dos grandes receptores) e **31** (Postfix real contra o simulador).
- Capítulo 10 ampliado com `posttls-finger` e `smtp_tls_loglevel`; capítulo 14 com os valores padrão de nova tentativa do Postfix.
- Logs, fila, DSN e falha de TLS dos capítulos 13, 14, 17, 19 e 26 agora vêm de **execução real** de Postfix, não de exemplos inventados.
- Laboratório reescrito: 22 testes, greylisting real por tripla, SIZE, transparência do ponto conforme a RFC e repetição sem reiniciar o servidor.
- Correções: `grep` no lugar de `rg`; `qshape` no RHEL; rotas por capítulo; apêndices; tabela de MX reescrita.

Sumário

FUNDAMENTOS

01 O percurso da mensagem e a evidência de cada etapa 04
02 Envelope e identificadores 04
03 MX e descoberta do próximo destino 05
04 Portas, conexão e ponto de observação 06

A SESSÃO SMTP

05 Um diálogo SMTP completo 07
06 Estados da sessão, respostas multilinha e transparência 07
07 Classes de resposta e etapa da falha 09
08 Códigos detalhados: classe, assunto e detalhe 09
09 EHLO e extensões que alteram a investigação 10
10 STARTTLS e verificação do canal 12
11 Submissão, AUTH e autorização de relay 14
12 Aceitação SMTP não é entrega na caixa postal 14

FILAS E EVIDÊNCIAS

13 Filas: estados, idade e concentração 16
14 Novas tentativas, expiração e duplicidade 17
15 Greylisting 19
16 Exigências dos grandes receptores 20
17 Devoluções e Delivery Status Notifications 21
18 Cabeçalhos e reconstrução dos saltos 22
19 Ler logs do Postfix sem perder a transação 23
20 Comandos de triagem e seus limites 23

CASOS COMENTADOS

21 Método de investigação por última etapa concluída 25
22 MX correto, conexão de saída bloqueada 25
23 Destinatário recusado e aceitação parcial 26
24 Mensagem adiada por condição temporária 27
25 Servidor aceitou, usuário não encontrou 27
26 TLS funciona no terminal, mas o MTA adia 28

LABORATÓRIO

27 Preparação e limites 29
28 Envelope, conteúdo e transparência 29
29 Falhas, aceitação parcial, SIZE e greylisting 30
30 STARTTLS com verificação real 30
31 Postfix real contra o simulador 31
32 Resultados verificados e o que eles não provam 32
33 Código do simulador 33
34 Código dos testes 37

OPERAÇÃO

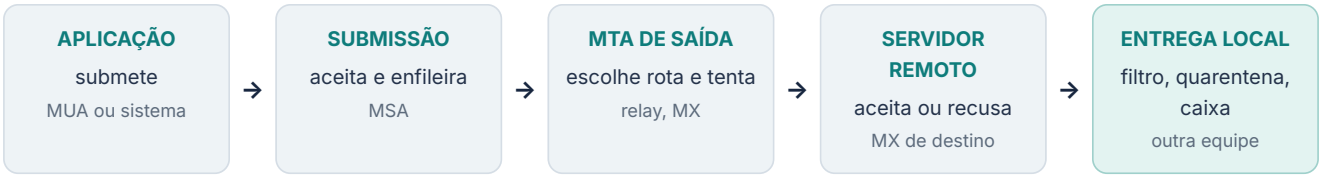
35 Relatório e linha do tempo 40
36 Mudanças seguras e critérios de aceite 40

APÊNDICES

A Exercícios de investigação 42
B Gabarito comentado 43
C Referência rápida por etapa 45
D Fontes oficiais 47
E Registro de edições e verificação 48

01 O percurso da mensagem e a evidência de cada etapa

Uma investigação de entrega acompanha uma mensagem concreta. "O e-mail não chegou" pode significar que a aplicação não submeteu, que o primeiro servidor recusou, que uma tentativa remota foi adiada ou que o destinatário recebeu em outra pasta. Comece por horário, envelope, destinatário e identificadores disponíveis.



Cada seta é um ponto onde a responsabilidade pode mudar de mãos. A investigação localiza a última seta comprovada.

Etapa	Pergunta	Evidência principal
Aplicação / cliente	Houve submissão ao servidor?	Log da aplicação e resposta SMTP recebida
Servidor de submissão	A mensagem foi aceita?	Resposta após o conteúdo e Queue-ID local
MTA de saída	Para onde tentou transferir?	Rota, MX, IP, porta e log da tentativa
Servidor remoto	Aceitou ou recusou a transação?	Código, etapa e texto da resposta
Entrega local	Onde o destinatário foi atendido?	Log de entrega, filtro ou caixa postal

1.1 Papéis

O cliente de e-mail é o MUA; o serviço de submissão, o MSA; o agente de transferência, o MTA. Um mesmo produto pode exercer vários papéis: identifique a função da conexão observada, não apenas a marca do servidor. Submissão e transferência aplicam políticas diferentes [3].

1.2 Um registro inicial útil

```
Incidente:       MAIL-2026-042
Horário:         2026-10-28T13:15:00-03:00
Origem:          ERP -> relay corporativo
MAIL FROM:       retornos@empresa.example
RCPT TO:         usuario@destino.example
Queue-ID local:  A1B2C3D4E5
Sintoma:         tentativa remota adiada
```

Dados fictícios. O objetivo é delimitar uma transação. Com vários destinatários, mantenha o estado de cada um: a mesma mensagem pode ter sido aceita para um endereço e recusada para outro.

02 Envelope e identificadores

O envelope orienta a transação SMTP; os cabeçalhos fazem parte do conteúdo transmitido. Eles podem ter valores diferentes sem que isso seja erro. Preserve ambos e explique suas funções.

Elemento	Função	Uso na investigação
MAIL FROM	Remetente do envelope / caminho de retorno	Devoluções e identidade SMTP
RCPT TO	Destinatário do envelope	Estado por destinatário
From:	Autor apresentado no conteúdo	Contexto e alinhamento de domínio
To: / Cc:	Destinatários exibidos	Não representam toda a lista do envelope
Message-ID:	Identificador da mensagem no conteúdo	Correlação, sem unicidade garantida
Queue-ID	Identificador local de fila	Correlacionar processos no mesmo servidor
Return-Path:	Caminho de retorno registrado na entrega	Interpretar com a cadeia de recebimento

```
MAIL FROM:<bounce@envios.empresa.example>
RCPT TO:<cliente@destino.example>
From: Atendimento <atendimento@empresa.example>
To: Cliente <cliente@destino.example>
Message-ID: <pedido-42@empresa.example>
```

From, To e Message-ID pertencem ao formato da mensagem definido pela RFC 5322 [2]. Não use o From visível para reconstruir o envelope: destinatários ocultos e expansões de lista tornam essa inferência frágil.

2.1 Correlação sem atalhos

Cada hop atribui seu próprio Queue-ID, e um filtro que reinjeta a mensagem pode gerar outro. Guarde pares de IDs, horário e resposta remota. Message-ID ajuda a procurar, mas pode faltar, repetir-se ou ser preservado em cópias distintas. Não agrupe incidentes pelo assunto: um ERP emite milhares de mensagens com o mesmo texto, cada uma com envelope e resultado próprios.

03 MX e descoberta do próximo destino

No encaminhamento convencional pela Internet, o domínio do destinatário orienta a busca de MX. Uma organização pode, porém, usar relay explícito ou regras de transporte que substituem esse caminho: antes de interpretar DNS, confirme a rota que o MTA realmente escolheu [1]. No Postfix, `relayhost` e `transport_maps` são os lugares a olhar; o capítulo 31 usa os dois.

```
dig destino.example. MX
dig mx1.destino.example. A
dig mx1.destino.example. AAAA
```

Publicação ilustrativa	Interpretação
`MX 10 mx1.destino.example.`	Preferido em relação ao 20: no MX, **número menor tem prioridade**.
`MX 20 mx2.destino.example.`	Alternativa, usada conforme a seleção e a disponibilidade do primeiro.
`MX 0 .`	Null MX: o domínio declara que não recebe e-mail [11].
Nenhum MX	A RFC 5321 (seção 5.1) prevê tratar o próprio domínio como destino implícito; não presuma Null MX.

O destino de MX é um nome, não um IP. Registros com a mesma preferência não indicam que um deles esteja desativado. Com vários hosts e famílias de IP, o log da tentativa é a evidência de qual endereço foi usado.

3.1 Ausência não é falha

Ausência de MX e erro temporário na consulta são coisas diferentes: o fallback previsto no SMTP não transforma um SERVFAIL em permissão para ignorar o DNS. Null MX é uma declaração explícita e não deve ser combinado com MX de serviço [1, 11].

3.2 O que coletar

RRset MX, A/AAAA dos alvos, resolvedor, TTL e horário; compare com as autoridades se houver divergência. Um MX de backup não garante entrega se não estiver preparado para aquele domínio ou não conhecer os destinatários válidos.

04 Portas, conexão e ponto de observação

A porta indica um serviço esperado, não prova sua configuração real. Investigue a partir do host ou da rede que faz o envio: um teste bem-sucedido no notebook não elimina bloqueio de saída no servidor do ERP.

Porta	Uso	Início da conexão
25/TCP	Transferência SMTP entre MTAs	SMTP; STARTTLS quando disponível
587/TCP	Submissão de mensagens	SMTP; política de TLS e autenticação
465/TCP	Submissão com TLS implícito	TLS antes do diálogo SMTP
1025/TCP	Simulador deste manual	Somente em loopback

A separação entre submissão e relay é da RFC 6409; a RFC 8314 trata TLS na submissão, incluindo TLS implícito [3, 4]. Não copie regras de autenticação de uma porta para outra sem conferir o serviço implantado.

4.1 Conexão estabelecida não é aceitação

Uma sessão TCP aberta ainda pode não receber banner, receber 421, falhar no EHLO ou ser recusada em RCPT. Registre a última etapa concluída. Ping não testa o serviço SMTP, e banner não testa o processamento da mensagem.

4.2 Matriz de comparação

Compare origem efetiva após NAT, endereço do destino, porta, família de IP e horário. Se só IPv6 falha, confira rota e serviço nessa família, e o PTR do IPv6 de saída (capítulo 16), antes de editar SPF. Se a rede bloqueia saída na 25, o MTA precisa do relay previsto pela arquitetura; mudar o MX do destinatário não resolve a origem bloqueada.

Não use conta real nem mensagem comercial para teste exploratório. Para observar o diálogo completo, use o laboratório local ou destinatários de teste definidos para a operação.

05 Um diálogo SMTP completo

Transcrição didática: C é o cliente, S é o servidor; os prefixos não são transmitidos. No DATA, uma linha contendo apenas ponto encerra o conteúdo.

```
S: 220 mx.destino.example ESMTP
C: EHLO relay.empresa.example
S: 250-mx.destino.example
S: 250 ENHANCEDSTATUSCODES
C: MAIL FROM:<retornos@empresa.example>
S: 250 2.1.0 Sender accepted
C: RCPT TO:<usuario@destino.example>
S: 250 2.1.5 Recipient accepted
C: DATA
S: 354 End data with <CRLF>.<CRLF>
C: From: Avisos <avisos@empresa.example>
C: To: Usuario <usuario@destino.example>
C: Date: Wed, 28 Oct 2026 13:15:00 -0300
C: Message-ID: <aula-42@empresa.example>
C: Subject: Mensagem de teste
C:
C: Conteudo de teste.
C: .
S: 250 2.0.0 Queued as R9S8T7
C: QUIT
S: 221 2.0.0 Bye
```

5.1 Onde está a evidência decisiva

O 250 de MAIL aceita o remetente; o de RCPT aceita aquele destinatário para a etapa seguinte; o 354 pede o conteúdo. A **resposta final após o conteúdo** é a evidência de aceitação nessa transferência: a partir dela, o receptor assume a responsabilidade pela mensagem [1]. O 221 encerra a sessão e não confirma entrega na caixa postal.

5.2 Como estudar a transcrição

Para cada resposta, escreva a qual comando ela responde. Depois troque mentalmente só o último 250 por 451: conexão, envelope e bytes teriam ocorrido, mas a transferência não teria sido concluída. Isso evita tratar "dados enviados" como "responsabilidade transferida".

06 Estados da sessão, respostas multilinha e transparência

SMTP é uma sequência de operações com estado. Um erro pode vir de comando na etapa errada, mesmo com endereço, DNS e rede corretos. Registre a ordem, não só a última linha de erro.

```
250-mx.destino.example
250-STARTTLS
250-SIZE 10485760
250 ENHANCEDSTATUSCODES
```

O hífen após o código indica continuação; o espaço indica a última linha. São quatro linhas de **uma** resposta, não respostas a quatro comandos [1].

Observação	Hipótese a testar
RCPT antes de MAIL	Sequência inválida do cliente (503)
DATA sem destinatário aceito	A transação não está pronta (503)
RSET antes de DATA	O envelope foi descartado
Comando após STARTTLS sem novo EHLO	O estado foi reiniciado; o servidor pode recusar (capítulo 10)
Desconexão durante DATA	Transferência incompleta ou conclusão incerta (capítulo 14)

6.1 Transparência do ponto

No DATA, uma linha de conteúdo que comece com ponto não pode ser confundida com o terminador. O cliente acrescenta um ponto a toda linha que começa com ponto (*dot-stuffing*), e o receptor, ao ver uma linha iniciada por ponto que tenha outros caracteres, remove esse primeiro ponto (RFC 5321, seção 4.5.2). O simulador desta edição segue essa regra, e o teste do capítulo 28 confirma que `.linha` e `..duas` chegam intactas.

Não cole linhas arbitrárias em sessões de produção "para ver se funciona": bibliotecas tratam fim de linha e transparência de forma mais confiável. O laboratório usa `smtplib`.

07 Classes de resposta e etapa da falha

O primeiro dígito do código SMTP dá a classe do resultado. Mas um número sem a etapa e o texto é evidência incompleta: o mesmo código genérico acompanha causas diferentes.

Classe / exemplo	Leitura	Ação investigativa
2xx / 250	Operação concluída	Identificar *qual* operação
3xx / 354	Continuação esperada	Enviar o conteúdo na sequência correta
4xx / 450, 451	Falha temporária	Preservar o estado e investigar a nova tentativa
421	Serviço indisponível ou encerrando	Correlacionar disponibilidade, limites de taxa e greylisting (capítulo 15)
5xx / 550, 554	Falha permanente da operação	Corrigir a causa antes de repetir

```
450 4.2.0 Temporary recipient failure
550 5.1.1 Unknown recipient
554 5.7.1 Message rejected by policy
```

A primeira representa falha temporária de destinatário; a segunda, destinatário inexistente; a terceira, política. Em uma coleta real, guarde a frase completa e o comando ao qual ela respondeu.

7.1 Consequência por destinatário

Uma recusa em RCPT atinge o destinatário daquela operação; outros podem ter sido aceitos. Uma resposta após DATA tem outro escopo e vale para o conjunto aceito naquela transação.

7.2 Não transformar 5xx em tentativa infinita

Repetir uma mensagem para destinatário incorreto reproduz a falha. Erro temporário pede fila e nova tentativa pelo sistema responsável. Uma aplicação que fala SMTP diretamente pode não ter fila durável: nesse caso, o 4xx vira perda silenciosa se a aplicação não guardar a mensagem (capítulo 14).

08 Códigos detalhados: classe, assunto e detalhe

Os enhanced status codes têm a forma X.Y.Z e acrescentam significado ao resultado [7, 8]. Guarde juntos o código de três dígitos, o código detalhado e o texto remoto, porque muitos sistemas usam códigos genéricos.

Código	Significado (RFC 3463)	Próxima evidência
2.0.0	Sucesso sem detalhe	Etapa e destino da aceitação
4.2.0	Estado de caixa postal indefinido, temporário	Estado do destinatário e log remoto (não prova caixa cheia)
4.4.1	Sem resposta do host	Conexão, rota e serviço
4.7.1	Entrega não autorizada, temporária	Greylisting, limite de taxa ou política (capítulos 15 e 16)
4.7.5	Falha de operação criptográfica	Verificação de TLS no MTA (capítulo 26; o Postfix usa este código)
5.1.1	Endereço de caixa de destino inválido	Endereço exato e cadastro
5.3.4	Mensagem grande demais para o sistema	Limites e tamanho efetivo
5.7.1	Entrega não autorizada, mensagem recusada	Política que gerou a recusa

8.1 Leitura de um evento real EXECUTADO

```
09:37:49 postfix/smtp[733]: DAC41C4554: to=<greylist@lab.test>, relay=localhost[127.0.0.1]:1025,
delay=0.17, delays=0.01/0.06/0.1/0, dsn=4.7.1, status=deferred (host localhost[127.0.0.1] said: 450
4.7.1 Greylisted, try again later (in reply to RCPT TO command))
```

Linha do Postfix de laboratório (capítulo 31). O código detalhado, o status e o texto descrevem uma recusa temporária em RCPT, de greylisting. A linha não demonstra problema de senha, de SPF ou de caixa cheia.

8.2 Por que a etapa importa

Um 5.7.1 em RCPT pode ser recusa de relay ou outra política; depois de DATA, pode ser decisão sobre o conteúdo. Não atribua todo 5.7.1 a SPF: procure o componente que decidiu e o que ele avaliou. O Gmail, por exemplo, usa 5.7.1 para IPv6 sem PTR (capítulo 16).

09 EHLO e extensões que alteram a investigação

O EHLO revela as capacidades daquela sessão. Extensão ausente não deve ser usada como se existisse, e as capacidades podem mudar depois do STARTTLS.

Extensão	O que observar	Referência
SIZE	Limite anunciado e tamanho declarado em MAIL FROM	RFC 1870 [20]
PIPELINING	Vários comandos antes das respostas	RFC 2920 [21]
CHUNKING	BDAT em vez do terminador de DATA	RFC 3030 [22]
8BITMIME	Conteúdo com octetos de 8 bits	RFC 6152 [30]
SMTPUTF8	Endereços e cabeçalhos internacionalizados	RFC 6531 [23]
DSN	Parâmetros de notificação no envelope	RFC 3461 [9]
AUTH / STARTTLS	Autenticação e proteção do canal	RFC 4954 / 3207 [6, 5]

9.1 Tamanho da mensagem

Anexos crescem com a codificação MIME. Compare o tamanho efetivo com os limites de cliente, relay e receptor. Quando o servidor anuncia SIZE, o cliente pode declarar o tamanho em MAIL FROM, e a recusa acontece antes do conteúdo: no laboratório, `MAIL FROM:<…> SIZE=2000000` recebe `552 5.3.4` (capítulo 29). O Postfix declara o tamanho automaticamente quando o servidor anuncia a extensão.

9.2 Capturas com PIPELINING e CHUNKING

Com PIPELINING, comandos vão antes das respostas correspondentes; com CHUNKING, procure BDAT e o tamanho dos blocos, não o ponto final. Um analisador que assume um único padrão produz cronologia enganosa [21, 22].

9.3 Internacionalização

8BITMIME e SMTPUTF8 não são equivalentes: o primeiro trata o corpo com octetos fora do ASCII; o segundo, endereços e informações internacionalizadas. Investigue as capacidades de cada hop, em vez de trocar a codificação do assunto [23, 30].

10 STARTTLS e verificação do canal

STARTTLS começa dentro de uma sessão SMTP e negocia TLS antes de continuar. A RFC 3207 exige que o servidor descarte o que sabia do cliente, que o cliente envie novo EHLO e que o servidor não anuncie STARTTLS de novo depois do handshake [5].

10.1 O que o openssl prova e o que não prova EXECUTADO

Duas execuções contra o simulador, cujo certificado é autoassinado e emitido para `localhost`:

```
$ openssl s_client -starttls smtp -connect 127.0.0.1:1025 -brief </dev/null
Can't use SSL_get_servername
depth=0 CN = localhost
verify error:num=18:self-signed certificate
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Ciphersuite: TLS_AES_256_GCM_SHA384
Peer certificate: CN = localhost
Hash used: SHA256

$ openssl s_client -starttls smtp -connect 127.0.0.1:1025 -CAfile cert.pem -verify_hostname nome-
errado.lab.test -verify_return_error -brief </dev/null
Can't use SSL_get_servername
depth=0 CN = localhost
verify error:num=62:hostname mismatch
404733B47D7F0000:error:0A000086:SSL routines:tls_post_process_server_certificate:certificate verify
failed:../ssl/statem/statem_clnt.c:1889:
```

Na primeira, mesmo com erro de verificação, aparece `CONNECTION ESTABLISHED` e uma cifra negociada. Sem `-verify_return_error`, o openssl continua e o teste "funciona". Na segunda, com a AC correta, um nome que o certificado não cobre e `-verify_return_error`, o handshake é interrompido. Para um alvo real:

```
openssl s_client -starttls smtp -connect mx.destino.example:25 \
    -servername mx.destino.example -verify_hostname mx.destino.example \
    -verify_return_error -CApath /etc/ssl/certs -brief </dev/null
```

`-servername` envia SNI, mas não verifica o nome; quem verifica é `-verify_hostname` [18]. O caminho das ACs varia: DEB `/etc/ssl/certs`; RPM `-CAfile /etc/pki/tls/certs/ca-bundle.crt`.

10.2 posttls-finger: testar como o Postfix testa

O openssl não reproduz a política do MTA. O `posttls-finger`, que acompanha o Postfix, conecta ao destino e verifica o certificado conforme um nível de segurança do próprio Postfix (`-l`, com a mesma semântica de `smtp_tls_security_level`) [32]. Ele usa as mesmas regras de busca de MX: `[host]:porta` desativa a busca de MX. Por padrão, nenhuma AC pública é confiável: informe `-F CAfile` ou `-P CApath/`. EXECUTADO

```
$ posttls-finger -c -h native -l secure -F cert.pem "[localhost]:1025"
posttls-finger: localhost[127.0.0.1]:1025: matched peername: localhost
posttls-finger: localhost[127.0.0.1]:1025: subject_CN=localhost, issuer_CN=localhost,
fingerprint=3E:D4:65:7D:7C..., pkey_fingerprint=D1:D7:72:26:58...
posttls-finger: Verified TLS connection established to localhost[127.0.0.1]:1025: TLSv1.3 with cipher
TLS_AES_256_GCM_SHA384 (256/256 bits) key-exchange X25519 server-signature RSA-PSS (2048 bits) server-
digest SHA256

$ posttls-finger -c -h native -l secure "[localhost]:1025"          # sem -F: nenhuma AC confiável
posttls-finger: certificate verification failed for localhost[127.0.0.1]:1025: self-signed certificate
posttls-finger: localhost[127.0.0.1]:1025: subject_CN=localhost, issuer_CN=localhost,
fingerprint=3E:D4:65:7D:7C..., pkey_fingerprint=D1:D7:72:26:58...
posttls-finger: Untrusted TLS connection established to localhost[127.0.0.1]:1025: TLSv1.3 with cipher
TLS_AES_256_GCM_SHA384 (256/256 bits) key-exchange X25519 server-signature RSA-PSS (2048 bits) server-
digest SHA256

$ posttls-finger -c -l secure -F cert.pem "[127.0.0.1]:1025" nome-errado.lab.test
posttls-finger: server certificate verification failed for 127.0.0.1[127.0.0.1]:1025: num=62:hostname
mismatch
posttls-finger: 127.0.0.1[127.0.0.1]:1025: subject_CN=localhost, issuer_CN=localhost,
fingerprint=3E:D4:65:7D:7C..., pkey_fingerprint=D1:D7:72:26:58...
posttls-finger: Untrusted TLS connection established to 127.0.0.1[127.0.0.1]:1025: TLSv1.3 with cipher
TLS_AES_256_GCM_SHA384 (256/256 bits) key-exchange X25519 server-signature RSA-PSS (2048 bits) server-
digest SHA256
```

As três saídas mostram o que o Postfix faria: conexão **Verified** quando AC e nome conferem; **Untrusted** por certificado autoassinado sem AC informada; e **Untrusted** por `hostname mismatch` quando o nome exigido (`nome-errado.lab.test`) não está no certificado. O código de saída do programa é 0 nos três casos: leia o texto, não o código. Pacote: DEB vem no `postfix` (conferido no 3.8.6 do Ubuntu 24.04); RPM confira na sua versão.

10.3 O registro do próprio MTA

Com `smtp_tls_loglevel = 1`, o cliente SMTP do Postfix registra um resumo de cada handshake. Os níveis 2 e superiores servem só para depuração, e o 4 é fortemente desaconselhado [15]. Linhas reais do laboratório EXECUTADO:

```
09:37:48 postfix/smtp[728]: Verified TLS connection established to localhost[127.0.0.1]:1025: TLSv1.3
with cipher TLS_AES_256_GCM_SHA384 (256/256 bits) key-exchange X25519 server-signature RSA-PSS (2048
bits) server-digest SHA256
09:37:49 postfix/smtp[739]: server certificate verification failed for 127.0.0.1[127.0.0.1]:1025:
num=62:hostname mismatch
09:37:49 postfix/smtp[739]: Untrusted TLS connection established to 127.0.0.1[127.0.0.1]:1025: TLSv1.3
with cipher TLS_AES_256_GCM_SHA384 (256/256 bits) key-exchange X25519 server-signature RSA-PSS (2048
bits) server-digest SHA256
```

TLS protege aquele salto. Não é criptografia ponta a ponta, não prova SPF, DKIM nem DMARC e não garante entrega na caixa.

11 Submissão, AUTH e autorização de relay

Submissão recebe mensagens de aplicações e usuários conforme a política da organização; transferência entre servidores atende outro papel. Não exija autenticação em todo MX público nem presuma que um servidor de submissão permita relay sem autenticação [3, 6].

Sintoma	Investigar
AUTH não anunciado	Porta, TLS, serviço e política (muitos servidores só anunciam AUTH depois do TLS)
Falha de autenticação	Mecanismo, identidade e backend
AUTH passa; MAIL recusado	Política de remetente da conta
AUTH passa; RCPT recusado	Destinatário, relay e restrições
Aplicação só funciona sem TLS	Configuração do cliente, confiança e modo de conexão

11.1 Autenticação não é autorização irrestrita

Uma conta pode autenticar e não estar autorizada a usar certo remetente, destinatário ou rota. Registre separadamente AUTH, MAIL, RCPT e o resultado final.

11.2 Modos TLS diferentes

STARTTLS em porta de TLS implícito, ou TLS implícito em porta SMTP simples, falha antes da autenticação. Confirme o modo do serviço antes de investigar a conta [4].

11.3 Segredos fora da evidência

Não copie senhas, tokens ou blocos AUTH para relatórios: Base64 não protege segredo. No Postfix, interprete restrições de relay no contexto da versão e da configuração ativa, e não remova restrições para testar uma hipótese [31].

12 Aceitação SMTP não é entrega na caixa postal

"Enviado" significa coisas diferentes na aplicação, no MTA e no destino. Use verbos precisos: submetido, aceito pelo relay, transferido ao próximo hop, entregue localmente, visualizado pelo usuário.

Evidência	O que sustenta	O que ainda não sustenta
250 em RCPT	Destinatário aceito nessa etapa	Conteúdo aceito
354	Receptor pronto para o conteúdo	Mensagem armazenada
250 após DATA	Mensagem aceita nessa transferência	Presença na caixa de entrada
`status=sent`	O próximo servidor aceitou	Último hop ou leitura
Log de entrega local	Ação do componente local	Leitura pelo usuário

12.1 Investigar depois da aceitação

Se o remoto aceitou, peça rastreamento pelo ID remoto, horário, envelope e Message-ID: filtros, quarentena, encaminhamento, aliases, regras de caixa e entrega. Em plataformas hospedadas, a ferramenta de rastreamento de mensagens do provedor cumpre esse papel. LMTP pode devolver resultado por destinatário depois do conteúdo; não aplique a ele a interpretação de uma transação SMTP [25].

12.2 Um limite honesto

Sem acesso ao servidor que aceitou, dá para demonstrar a transferência, não o destino interno. Registre: "aceito por X, ID Y, às Z; entrega local não verificada". É mais útil do que afirmar sucesso completo ou perda definitiva sem evidência.

13 Filas: estados, idade e concentração

A fila registra o trabalho pendente do MTA, não apenas mensagens "presas". Analise quantidade, idade, destinatários, domínios e motivos de adiamento. Um pico de mensagens recentes significa algo diferente de um pequeno conjunto que envelhece há horas.

Fila Postfix	Interpretação
maildrop	Submissão local aguardando o `pickup`
incoming	Mensagens recebidas aguardando o gerenciador de fila
active	Mensagens selecionadas para tentativa de entrega
deferred	Entregas adiadas para tentativas posteriores
hold	Mensagens retidas até intervenção

O Postfix separa recebimento, preparação, gerenciamento da fila e agentes de entrega: `smtpd` recebe sessões, `pickup` lê a submissão local, `cleanup` prepara, `qmgr` organiza e `smtp` é o cliente de transferência [14, 29]. O nome do processo no log localiza a etapa.

13.1 Coleta inicial EXECUTADO

Fila real do Postfix de laboratório (capítulo 31), com duas mensagens adiadas por motivos diferentes:

```
$ postqueue -p
-Queue ID-  --Size-- ----Arrival Time---- -Sender/Recipient-------
E6E05C4559      289 Thu Oct 29 09:37:48  aceito@lab.test
                                             (Server certificate not verified)
                                         aceito@tlserro.lab.test

E2EFCC4558      299 Thu Oct 29 09:37:48  aceito@lab.test
(host localhost[127.0.0.1] said: 450 4.2.0 Temporary recipient failure (in reply to RCPT TO command))
                                         temporario@lab.test

-- 0 Kbytes in 2 Requests.

$ postqueue -j (uma linha por mensagem)
{"queue_name": "deferred", "queue_id": "E6E05C4559", "arrival_time": 1793277468, "message_size": 289,
"forced_expire": false, "sender": "aceito@lab.test", "recipients": [{"address":
"aceito@tlserro.lab.test", "delay_reason": "Server certificate not verified"}]}
{"queue_name": "deferred", "queue_id": "E2EFCC4558", "arrival_time": 1793277468, "message_size": 299,
"forced_expire": false, "sender": "aceito@lab.test", "recipients": [{"address": "temporario@lab.test",
"delay_reason": "host localhost[127.0.0.1] said: 450 4.2.0 Temporary recipient failure (in reply to
RCPT TO command)"}]}

$ qshape deferred
                                      T  5 10 20 40 80 160 320 640 1280 1280+
                                TOTAL  2  2  0  0  0  0   0   0   0    0     0
                              lab.test  1  1  0  0  0  0   0   0   0    0     0
                        tlserro.lab.test  1  1  0  0  0  0   0   0   0    0     0
```

- `postqueue -p` mostra o motivo atual de cada destinatário adiado.
- `postqueue -j` produz uma linha JSON por mensagem, com `delay_reason` por destinatário; existe a partir do Postfix 3.1 [12].
- `qshape deferred` agrupa por domínio e por idade, em minutos. DEB vem no pacote `postfix`; RPM fica no pacote separado `postfix-perl-scripts` [35].

`-p` e `-j` apenas consultam. Outras opções do `postqueue`, como `-f`, `-i` e `-s`, agendam entregas imediatas e alteram o comportamento [12].

13.2 Pergunta de causa

Se quase toda a fila aponta para um destino, investigue esse caminho. Se muitos destinos falham, procure um componente compartilhado: DNS, conectividade, relay obrigatório, recursos locais ou política de TLS. Se há muitas mensagens em hold, descubra quem as reteve e por quê. Liberar ou apagar só faz sentido depois de entender a causa e o impacto.

14 Novas tentativas, expiração e duplicidade

Uma falha temporária mantém o trabalho pendente para nova tentativa. Intervalo e prazo dependem da implementação e da configuração: não prometa um tempo universal de reenvio ou de devolução com base só no código 4xx.

14.1 Valores padrão do Postfix EXECUTADO

```
$ postconf -d queue_run_delay minimal_backoff_time maximal_backoff_time \
      maximal_queue_lifetime bounce_queue_lifetime
queue_run_delay = 300s
minimal_backoff_time = 300s
maximal_backoff_time = 4000s
maximal_queue_lifetime = 5d
bounce_queue_lifetime = 5d
```

Parâmetro	Significado [15]
`queue_run_delay`	Intervalo entre varreduras da fila deferred pelo gerenciador
`minimal_backoff_time`	Tempo mínimo entre tentativas de uma mensagem adiada
`maximal_backoff_time`	Tempo máximo entre tentativas
`maximal_queue_lifetime`	Idade a partir da qual uma falha temporária vira devolução
`bounce_queue_lifetime`	O mesmo, para as próprias notificações de devolução

Na prática, com os padrões: uma mensagem adiada é tentada de novo depois de cerca de 5 minutos, o intervalo cresce até pouco mais de uma hora, e após 5 dias na fila a falha temporária vira devolução. Use `postconf` sem `-d` para ver os valores efetivos do seu servidor; regras por transporte também influenciam.

14.2 Uma nova tentativa real EXECUTADO

```
09:37:49 postfix/smtp[733]: DAC41C4554: to=<greylist@lab.test>, relay=localhost[127.0.0.1]:1025,
delay=0.17, delays=0.01/0.06/0.1/0, dsn=4.7.1, status=deferred (host localhost[127.0.0.1] said: 450
4.7.1 Greylisted, try again later (in reply to RCPT TO command))
09:37:58 postfix/smtp[733]: DAC41C4554: to=<greylist@lab.test>, relay=localhost[127.0.0.1]:1025,
delay=9.4, delays=9.3/0/0.09/0, dsn=2.0.0, status=sent (250 2.0.0 Stored locally as LAB86918478AC70)
```

O laboratório reduziu os tempos para 10 s. A primeira tentativa recebeu 450; a segunda, 9 segundos depois, foi aceita. No `delays=9.3/0/0.09/0` da segunda linha, o primeiro valor inclui o tempo que a mensagem passou na fila adiada (capítulo 19).

14.3 O caso ambíguo do último 250

Se o receptor armazenou a mensagem e respondeu sucesso, mas a conexão caiu antes que o remetente lesse a resposta, os dois lados têm visões diferentes, e uma nova tentativa gera duplicidade. Correlacione os logs dos dois hops; não conclua que a mensagem "não foi enviada" porque a aplicação recebeu timeout [1].

14.4 Aplicação sem fila própria

Um script que fala SMTP diretamente apenas recebe uma exceção ou um código. Isso não equivale a um MTA com fila durável. Identifique quem guarda e repete o trabalho; sistemas transacionais precisam tratar resultado incerto e evitar reenvio indiscriminado. Forçar a fila inteira repetidas vezes gera carga e repete o mesmo erro; depois de corrigir a causa, acompanhe a recuperação ou faça tentativa direcionada.

15 Greylisting NOVO

Greylisting é a prática de oferecer serviço temporariamente degradado a clientes desconhecidos como medida antiabuso: o receptor responde com falha temporária (4xx) a uma origem que ainda não conhece e aceita quando ela tenta de novo depois de um intervalo [33]. É uma das causas mais comuns de "mensagem adiada" para destinatários novos.

15.1 Como funciona

- A implementação clássica guarda uma tripla: IP do cliente, remetente e destinatário. Na primeira vez, recusa temporariamente; se a mesma tripla voltar depois do atraso mínimo, aceita e passa a lembrar dela.
- A RFC 6647 recomenda que a recusa use 421 ou 450 [33], e a resposta pode vir já na conexão ou em comandos posteriores, como RCPT.
- Só funciona porque remetentes legítimos repetem a tentativa: a RFC afirma que um cliente precisa saber tentar de novo para ser considerado plenamente capaz, e clientes que não repetem veem falha definitiva [33].

15.2 Sintomas típicos

Observação	Leitura
450/451 4.7.1 com texto como "Greylisted" ou "try again later" para destinatário novo	Greylisting provável; confirme com o texto e com a tentativa seguinte
Atraso de minutos na primeira mensagem a um parceiro, sem repetição depois	Comportamento esperado; a tripla passou a ser conhecida
Mensagem nunca chega e a aplicação não tem fila	A aplicação não repete: a culpa não é do greylisting (capítulo 14.4)
Atraso grande, de uma hora ou mais	O intervalo de tentativa do remetente é longo; a RFC cita esse efeito sobre mensagens urgentes [33]
Atraso que se repete sempre	O remetente sai por IPs diferentes a cada tentativa (pool de envio), e a tripla nunca se repete

15.3 Como confirmar e o que fazer

01 Leia o texto completo da resposta: muitos receptores dizem explicitamente que é greylisting.

02 Verifique no log se houve segunda tentativa e com que IP de origem saiu.

03 Se você opera o MTA remetente, garanta que ele repete normalmente (padrões do capítulo 14). Não reduza os intervalos só para vencer greylisting.

04 Se a aplicação fala SMTP diretamente, faça-a submeter a um MTA com fila. Esse é o conserto, não pedir exceção ao destino.

05 Se você opera o greylisting, mantenha lista de exceção para grandes remetentes que usam pools de IP, conforme sua política.

O laboratório implementa greylisting real por tripla com atraso de 3 segundos, e o Postfix do capítulo 31 o atravessa sozinho: veja o log do capítulo 14.2.

16 Exigências dos grandes receptores NOVO

Grandes provedores publicam exigências para aceitar mensagens. Recusas por essas regras costumam aparecer como 4.7.x ou 5.7.x e caem na vala comum de "política" se o texto não for lido. As do Gmail, por exemplo [34]:

Quem	Exigência
Todos os remetentes	SPF ou DKIM no domínio de envio; DNS direto e reverso (PTR) válidos para o domínio ou IP de envio; TLS na transmissão; taxa de spam abaixo de 0,3% no Postmaster Tools; mensagens no formato da RFC 5322; não imitar cabeçalhos From do Gmail.
Remetentes em massa (mais de 5.000 mensagens por dia)	SPF **e** DKIM, mais DMARC (a política pode ser `none`); From alinhado ao domínio do SPF ou do DKIM; cancelamento de inscrição com um clique e link visível no corpo.

16.1 IPv6 sem PTR

O Gmail recusa mensagens por IPv6 sem PTR adequado com a resposta `550-5.7.1: Message does not meet IPv6 sending guidelines regarding PTR records and authentication`, e recomenda confirmar o PTR do IPv6 com o provedor de envio [34]. É o exemplo perfeito do capítulo 4.2: só a família IPv6 falha, e a causa não está no MX nem no SPF.

16.2 Como investigar

01 Copie o texto completo: grandes provedores costumam incluir um link para a regra violada.

02 Identifique o IP de saída efetivo e a família, e verifique PTR e DNS direto desse IP.

03 Para exigências de autenticação, use os manuais de SPF e DMARC desta série.

04 Consulte sempre a página oficial do provedor no momento do incidente: essas regras mudam, e outros provedores publicam exigências próprias.

17 Devoluções e Delivery Status Notifications

Uma DSN descreve o resultado de entrega por destinatário; não é confirmação de leitura. Além do texto ao usuário, examine a parte estruturada `message/delivery-status` [9, 10].

17.1 Uma DSN real EXECUTADO

Gerada pelo Postfix de laboratório quando o simulador recusou `desconhecido@lab.test` com 550, e entregue de volta ao remetente do envelope (trechos):

```
From: Mail Delivery System <MAILER-DAEMON@relay.lab.test>
Subject: Undelivered Mail Returned to Sender
To: aceito@lab.test
Auto-Submitted: auto-replied
Content-Type: multipart/report; report-type=delivery-status;
   ...
--D61BBC4551.1793277469/relay.lab.test
Content-Description: Delivery report
Content-Type: message/delivery-status

Reporting-MTA: dns; relay.lab.test
X-Postfix-Queue-ID: D61BBC4551
X-Postfix-Sender: rfc822; aceito@lab.test
Arrival-Date: Thu, 29 Oct 2026 09:37:48 -0300 (-03)

Final-Recipient: rfc822; desconhecido@lab.test
Original-Recipient: rfc822;desconhecido@lab.test
Action: failed
Status: 5.1.1
Remote-MTA: dns; localhost
Diagnostic-Code: smtp; 550 5.1.1 Unknown recipient
   ... (segue a mensagem original, como message/rfc822)
```

Campo	Leitura
Reporting-MTA	Quem gerou a notificação: o relay que aceitou e depois falhou
X-Postfix-Queue-ID	Queue-ID da mensagem original no relay; liga a DSN ao log (capítulo 19)
Final-Recipient	Destinatário a que o resultado se refere
Action / Status	Ação (failed, delayed, delivered…) e código detalhado
Remote-MTA / Diagnostic-Code	Quem recusou e a resposta de transporte original

17.2 Rejeição imediata e falha posterior

Se a recusa acontece durante a sessão, o cliente SMTP recebe a resposta e não há DSN do receptor. Se um relay já aceitou e falha depois, é **ele** quem gera a notificação ao caminho de retorno, como no exemplo. Reconstrua quem aceitou e quem notificou.

17.3 Envelope vazio

Notificações usam caminho de retorno vazio para evitar laços. No laboratório, o simulador registrou a DSN chegando com `MAIL FROM:<>`. Não trate a ausência de endereço como defeito; relacione com o estudo de SPF/HELO do manual de SPF [1].

Uma DSN pode ser falsificada ou não chegar a quem se espera: corrobore com logs e não use "delivered" como prova de leitura humana.

18 Cabeçalhos e reconstrução dos saltos

Cada servidor acrescenta um `Received` no topo. Para reconstruir a cronologia, leia de baixo para cima, mas estabeleça antes a fronteira de confiança: linhas anteriores à sua infraestrutura podem ter sido escritas por terceiros.

```
Received: from relay.empresa.example (relay [192.0.2.10])
    by mx.destino.example with ESMTPS id R9S8T7
    for <usuario@destino.example>;
    Wed, 28 Oct 2026 13:15:03 -0300
Received: from app.interno.example (app [10.0.0.20])
    by relay.empresa.example with ESMTP id A1B2C3D4E5;
    Wed, 28 Oct 2026 13:15:00 -0300
Message-ID: <pedido-42@empresa.example>
```

Exemplo construído: dois saltos, dois identificadores locais. `ESMTPS` indica sessão com TLS naquele salto. Relógios e fusos errados distorcem cálculos: não some diferenças de horário sem verificar sincronização [2].

18.1 Correlacionar com o log

Procure o ID local no servidor que o gerou. A resposta do hop seguinte traz o ID dele (no laboratório, `Stored locally as LAB…`) e faz a ponte entre sistemas. Use envelope e horário para evitar colisões de busca.

18.2 Authentication-Results é outra evidência

Descreve resultados de autenticação e só vale como evidência se vier de componente confiável. Não demonstra entrega na caixa; separe spf/dkim/dmarc do resultado de transporte e da disposição local [24].

18.3 Falha antes da entrega

Uma mensagem recusada durante o SMTP não existe na caixa do destinatário para extração de cabeçalhos: o log da sessão nos dois lados é o caminho.

19 Ler logs do Postfix sem perder a transação

O log deve ser lido como uma sequência correlacionada pelo Queue-ID. Formato e destino variam por distribuição e configuração; confirme onde o Postfix está registrando e a janela de tempo do relato [16].

19.1 Duas transações reais EXECUTADO

```
09:37:48 postfix/pickup[692]: D2388C4553: uid=0 from=<aceito@lab.test>
09:37:48 postfix/cleanup[712]: D2388C4553: message-id=<manual-aceito@lab.test>
09:37:48 postfix/qmgr[694]: D2388C4553: from=<aceito@lab.test>, size=287, nrcpt=1 (queue active)
09:37:48 postfix/pickup[692]: D61BBC4551: uid=0 from=<aceito@lab.test>
09:37:48 postfix/cleanup[712]: D61BBC4551: message-id=<manual-desconhecido@lab.test>
09:37:48 postfix/qmgr[694]: D61BBC4551: from=<aceito@lab.test>, size=305, nrcpt=1 (queue active)
09:37:48 postfix/smtp[728]: Verified TLS connection established to localhost[127.0.0.1]:1025: TLSv1.3
with cipher TLS_AES_256_GCM_SHA384 (256/256 bits) key-exchange X25519 server-signature RSA-PSS (2048
bits) server-digest SHA256
09:37:48 postfix/smtp[718]: Verified TLS connection established to localhost[127.0.0.1]:1025: TLSv1.3
with cipher TLS_AES_256_GCM_SHA384 (256/256 bits) key-exchange X25519 server-signature RSA-PSS (2048
bits) server-digest SHA256
09:37:49 postfix/smtp[718]: D2388C4553: to=<aceito@lab.test>, relay=localhost[127.0.0.1]:1025,
delay=0.2, delays=0.03/0.05/0.12/0, dsn=2.0.0, status=sent (250 2.0.0 Stored locally as
LAB21855A81EE14)
09:37:49 postfix/qmgr[694]: D2388C4553: removed
09:37:49 postfix/smtp[728]: D61BBC4551: to=<desconhecido@lab.test>, relay=localhost[127.0.0.1]:1025,
delay=0.18, delays=0.02/0.05/0.1/0.01, dsn=5.1.1, status=bounced (host localhost[127.0.0.1] said: 550
5.1.1 Unknown recipient (in reply to RCPT TO command))
09:37:49 postfix/cleanup[712]: 0CE90C455E: message-id=<20261029123749.0CE90C455E@relay.lab.test>
09:37:49 postfix/qmgr[694]: 0CE90C455E: from=<>, size=2247, nrcpt=1 (queue active)
09:37:49 postfix/bounce[742]: D61BBC4551: sender non-delivery notification: 0CE90C455E
09:37:49 postfix/qmgr[694]: D61BBC4551: removed
09:37:49 postfix/smtp[718]: 0CE90C455E: to=<aceito@lab.test>, relay=localhost[127.0.0.1]:1025,
delay=0.11, delays=0/0/0.1/0, dsn=2.0.0, status=sent (250 2.0.0 Stored locally as LABF24D2C4F7394)
09:37:49 postfix/qmgr[694]: 0CE90C455E: removed
```

19.2 O que a sequência demonstra

- **Primeira mensagem:** `pickup` leu a submissão local, `cleanup` registrou o Message-ID, `qmgr` colocou na fila ativa, `smtp` estabeleceu TLS verificado e transferiu; a resposta remota traz o ID `LAB…`; `removed` encerra o trabalho local, sem provar leitura.
- **Segunda mensagem:** recusa permanente em RCPT (`status=bounced`). O `bounce` gerou a DSN com novo Queue-ID, que entrou na fila com `from=<>` e foi entregue.

19.3 Decompor o atraso

No formato `delays=a/b/c/d`, segundo a documentação do Postfix [15]: **a**, da chegada até a última entrada na fila ativa (inclui o tempo na fila adiada); **b**, da fila ativa até o início do estabelecimento da conexão; **c**, estabelecimento da conexão — em SMTP, tudo antes do MAIL FROM, incluindo DNS, TCP, EHLO e STARTTLS numa conexão nova; **d**, transmissão da mensagem.

Não transforme "c alto" em "DNS lento": use a decomposição para escolher a próxima coleta, confrontando logs de TLS, latência e comportamento do remoto.

20 Comandos de triagem e seus limites

Comece com leitura. Copie os resultados antes de qualquer operação que altere a fila ou a configuração.

```
postconf mail_version
postconf -n                     # parâmetros fora do padrão
postconf maillog_file           # destino do log, se configurado (Postfix 3.4+)
postqueue -p
postqueue -j
sudo postcat -q A1B2C3D4E5      # conteúdo e envelope de um item da fila
sudo grep -E 'A1B2C3D4E5|R9S8T7' /var/log/mail.log      # DEB
sudo grep -E 'A1B2C3D4E5|R9S8T7' /var/log/maillog       # RPM
journalctl --since '2026-10-28 13:00' --until '2026-10-28 14:00' --no-pager | grep postfix
```

20.1 Como usar cada resultado

`postconf -n` mostra parâmetros fora do padrão, não todos os serviços e overrides do `master.cf`; consulte valores específicos quando necessário. `postcat -q` expõe mensagem e envelope, e só funciona enquanto o item está na fila [13]. O local do log depende da distribuição e da configuração: `/var/log/mail.log` é o costume em Debian/Ubuntu, `/var/log/maillog` em RHEL, o journal em sistemas sem syslog, ou o arquivo de `maillog_file` [15, 16]. Não conclua ausência de atividade a partir de um filtro que não corresponde à implantação.

20.2 O que não é consulta

Flush, requeue, liberação de hold, exclusão de fila e reload alteram o comportamento: podem repetir entregas, aumentar a carga ou apagar a evidência necessária. Anonimize dados antes de compartilhar.

21 Método de investigação por última etapa concluída

01 **Obter uma transação concreta.** Horário com fuso, origem, MAIL FROM, RCPT TO, Message-ID e Queue-ID. Confirme se o relato é falha de submissão, de transferência ou de localização na caixa.

02 **Descobrir a rota efetiva.** Relay explícito, regra de transporte ou MX. Registre host e IP tentados, comparando a consulta DNS contemporânea com o destino registrado no log, sem substituir a história por uma consulta atual.

03 **Localizar a última etapa confirmada.** Conexão TCP, banner, EHLO, STARTTLS, AUTH, MAIL, RCPT, DATA ou resultado final. Anote a primeira etapa que falhou e quem respondeu.

04 **Classificar o estado por destinatário.** Temporário, permanente, aceito ou desconhecido. Em aceitação parcial, mantenha as duas listas; em timeout após o conteúdo, registre a possibilidade de resultado incerto.

05 **Testar uma hipótese por vez.** Exemplo: "o certificado não cobre o nome do MX". Defina o teste que verifica nome e cadeia (de preferência com `posttls-finger`), preserve a saída e confronte com a política do MTA.

06 **Corrigir com escopo limitado.** A menor mudança que trate a causa confirmada, com validação e reversão definidas, sem segredos no relatório.

07 **Validar o fluxo original.** Repita o teste permitido, confirme o próximo hop e, se o objetivo era a caixa, obtenha evidência dessa etapa. Verifique regressões e encerre com causa, evidência, correção e limite de visibilidade.

22 Caso: MX correto, conexão de saída bloqueada

Cenário fictício. O ERP aceita pedidos, mas mensagens para vários domínios ficam adiadas no relay. MX e A/AAAA respondem. O log mostra timeout ao conectar à porta 25 de destinos diferentes. Um teste no notebook funciona.

Evidência	Interpretação
DNS responde	A descoberta não é a falha observada
Muitos destinos afetados	Componente compartilhado
Timeout antes do banner (`dsn=4.4.1`)	Falha anterior ao diálogo SMTP
Notebook funciona	O caminho do notebook não representa o relay
Servidor mudou de rede	Rota, NAT e política de saída

```
to=<usuario@destino.example>, dsn=4.4.1, status=deferred
(connect to mx.destino.example[192.0.2.25]:25: Connection timed out)   # exemplo construído
```

Confirmação: a equipe de rede mostra que a nova rede do relay bloqueia saída TCP/25, com logs da janela, IP de NAT e destinos coincidentes com as tentativas. Essa correlação sustenta a causa; o timeout sozinho não a provaria.

Correção: restaurar a saída autorizada ou usar o relay intermediário previsto. Não trocar MX de destinatários nem adicionar IPs ao SPF. **Aceite:** novas tentativas aceitas pelos destinos, fila envelhecida diminuindo sem duplicações, e horários da liberação e da primeira transferência registrados.

23 Caso: destinatário recusado e aceitação parcial

```
C: MAIL FROM:<retornos@empresa.example>
S: 250 2.1.0 OK
C: RCPT TO:<ana@destino.example>
S: 250 2.1.5 OK
C: RCPT TO:<endereco-antigo@destino.example>
S: 550 5.1.1 Unknown recipient
C: DATA
S: 354 Continue
   ...conteúdo...
S: 250 2.0.0 Queued as R9S8T7
```

A mensagem não tem um estado único: Ana foi aceita; o endereço antigo, não. Verifique o cadastro ou a regra de expansão antes de repetir para o recusado.

Risco de reenvio global: se a aplicação tratar a recusa parcial como falha total e reenviar a lista inteira, Ana recebe cópia duplicada. A aplicação precisa guardar aceitos e recusados separadamente, ou delegar isso a um MTA.

No laboratório: `sendmail()` do Python devolve um dicionário só com os recusados quando ao menos um é aceito, e lança `SMTPRecipientsRefused` quando todos são recusados [19]. A ausência de exceção não significa que toda a lista foi aceita (capítulo 29). No Postfix, a recusa gera DSN apenas para o destinatário recusado (capítulo 17).

24 Caso: mensagem adiada por condição temporária

Cenário fictício. O receptor responde `450 4.7.1` em RCPT para um parceiro novo; o relay mantém a mensagem e uma tentativa posterior é aceita.

Horário	Evento	Estado
13:00	RCPT recebe 450 4.7.1 "Greylisted"	Destinatário não aceito nessa tentativa
13:00	Fila local mostra deferred	Trabalho permanece com o remetente
13:05	Nova tentativa (padrão do Postfix: 300 s)	Aceita; transferência concluída

24.1 Duas hipóteses principais

- **Greylisting** (capítulo 15): texto explícito, destinatário ou parceiro novo, aceito na tentativa seguinte. Esperado; não exige correção se o MTA repete.
- **Condição temporária do destino** (caixa, armazenamento, limite de taxa): o código 4.2.x não prova caixa cheia nem informa quando vai passar; o texto e os logs remotos especificam.

24.2 Investigação e validação

Guarde as respostas completas, a idade na fila e o IP de saída de cada tentativa: se o IP muda, o greylisting pode nunca liberar. Verifique os parâmetros de tentativa do seu MTA. Não reenvie manualmente enquanto a original está pendente. A tentativa bem-sucedida precisa corresponder à entrada original (IDs, envelope e horário); se o usuário relatar duplicidade depois, procure reenvio paralelo da aplicação.

O capítulo 14.2 mostra esse caso acontecendo de verdade no Postfix de laboratório.

25 Caso: servidor aceitou, usuário não encontrou

O MTA de saída registra `status=sent` para o MX remoto, com 250 após o conteúdo e ID remoto R9S8T7. O destinatário diz que não recebeu. Isso pede investigação depois da transferência, não mudança no envio.

Resultado remoto	Próxima ação
Quarentena	Motivo, política e liberação autorizada
Entregue em outra pasta	Regras e filtros da caixa
Encaminhamento	Continuar no próximo destino
Falha interna após aceitação	Notificação e responsabilidade local
Sem acesso ao rastreamento	Declarar o limite da conclusão

Peça busca por R9S8T7 no receptor, com horário, envelope e Message-ID; em plataformas hospedadas, use a ferramenta de rastreamento de mensagens do provedor. Se houver gateway separado do armazenamento, siga o novo hop e o novo ID.

Causa construída: o gateway aceitou e aplicou quarentena; a equipe confirma a regra. O remetente não perdeu a mensagem; a disposição local impediu a entrada na caixa. Ajuste a política só se o fluxo for legítimo, valide com mensagem de teste e não faça allowlist ampla como reflexo. Uma mensagem pode autenticar perfeitamente e ainda ir para a quarentena: SPF, DKIM e DMARC não substituem as outras decisões do receptor.

26 Caso: TLS funciona no terminal, mas o MTA adia

Um `openssl s_client` simples negocia uma cifra; o MTA, porém, exige identidade verificada para o destino e adia a entrega. O certificado não cobre o nome exigido, e o primeiro teste não verificava o nome. **Nesta edição, o caso foi reproduzido de verdade** no laboratório (capítulo 31) `EXECUTADO`:

```
09:37:48 postfix/pickup[692]: E6E05C4559: uid=0 from=<aceito@lab.test>
09:37:48 postfix/cleanup[712]: E6E05C4559: message-id=<manual-tls@lab.test>
09:37:48 postfix/qmgr[694]: E6E05C4559: from=<aceito@lab.test>, size=289, nrcpt=1 (queue active)
09:37:49 postfix/smtp[739]: server certificate verification failed for 127.0.0.1[127.0.0.1]:1025:
num=62:hostname mismatch
09:37:49 postfix/smtp[739]: Untrusted TLS connection established to 127.0.0.1[127.0.0.1]:1025: TLSv1.3
with cipher TLS_AES_256_GCM_SHA384 (256/256 bits) key-exchange X25519 server-signature RSA-PSS (2048
bits) server-digest SHA256
09:37:49 postfix/smtp[739]: E6E05C4559: to=<aceito@tlserro.lab.test>, relay=127.0.0.1[127.0.0.1]:1025,
delay=0.1, delays=0/0.04/0.05/0, dsn=4.7.5, status=deferred (Server certificate not verified)
09:37:58 postfix/qmgr[694]: E6E05C4559: from=<aceito@lab.test>, size=289, nrcpt=1 (queue active)
09:37:58 postfix/smtp[728]: E6E05C4559: to=<aceito@tlserro.lab.test>, relay=127.0.0.1[127.0.0.1]:1025,
delay=9.3, delays=9.2/0/0.05/0, dsn=4.7.5, status=deferred (Server certificate not verified)
```

O mesmo simulador, com o mesmo certificado, foi aceito como *Verified* quando o nome exigido era `localhost`, e recusado quando a política exigia `nome-errado.lab.test`. A conexão TLS foi estabelecida (*Untrusted*), mas a política `secure` não aceita identidade não verificada, e a mensagem ficou adiada com `dsn=4.7.5`. O openssl sem verificação, no capítulo 10.1, "passava" no mesmo cenário.

26.1 Investigar com o objetivo certo

01 Leia a política efetiva: `postconf smtp_tls_security_level smtp_tls_policy_maps` e as regras por destino.

02 Reproduza com `posttls-finger -l <nível>` e o mesmo nome exigido, a partir do próprio MTA (capítulo 10.2).

03 Inspecione cadeia, validade e nomes do certificado (SAN).

04 Se houver MTA-STS (RFC 8461) ou DANE (RFC 7672) no destino, lembre que são mecanismos distintos, com caches e dependências próprias [26, 27]; relatórios TLSRPT ajudam, mas não substituem o log de uma mensagem [28].

Causa e correção do exemplo: a renovação omitiu um nome usado pelo MX. Publicar o certificado correto e retestar com a política original. Não reduza permanentemente a exigência de verificação só para esvaziar a fila. **Aceite:** sucesso no MTA original, com a política original, para o mesmo destino, registrando MX, endereço, resultado de confiança e horário.

27 Laboratório: preparação e limites

Arquivo	Função
`preparar_tls.py`	Gera certificado e chave locais (localhost e 127.0.0.1), válidos por 7 dias
`servidor_smtp.py`	Simulador SMTP em 127.0.0.1:1025 com STARTTLS real
`testar_smtp.py`	22 testes com `smtplib`; pode ser repetido sem reiniciar o servidor
`main.cf.lab`, `transport.lab`, `tls_policy.lab`	Configuração de Postfix usada no capítulo 31
`enviar_testes_postfix.sh`	Submete cinco mensagens ao Postfix de laboratório

```
python3 preparar_tls.py
python3 servidor_smtp.py
# Em outro terminal, na mesma pasta:
python3 testar_smtp.py
```

Requisitos: Python 3.11 ou superior, sem pacotes externos, e OpenSSL 1.1.1 ou superior. Esta edição usou Python 3.13.18 e OpenSSL 3.0.13. O script usa `-noenc` no OpenSSL 3, onde `-nodes` é obsoleto [36]. A chave recebe permissão 600; não instale o certificado na confiança global nem reutilize a chave.

27.1 Destinatários do simulador

Destinatário	Comportamento
`aceito@lab.test`	RCPT e conteúdo aceitos; grava `LAB*.eml`
`temporario@lab.test`	RCPT recebe `450 4.2.0`
`greylist@lab.test`	Greylisting por tripla: `450 4.7.1` até passar o atraso mínimo (3 s, variável `SMTP_LAB_GREY`)
`dados-temp@lab.test`	RCPT aceito; conteúdo recebe `451 4.3.0`
`dados-perm@lab.test`	RCPT aceito; conteúdo recebe `554 5.7.1`
qualquer outro	RCPT recebe `550 5.1.1`

O simulador anuncia `SIZE 1048576`, aplica a regra de transparência da RFC 5321, exige novo EHLO após STARTTLS e descarta texto em claro recebido após o comando STARTTLS. Não implementa fila, DSN, AUTH, antispam, entrega em caixa, DANE ou MTA-STS: não é um MTA de produção.

28 Laboratório: envelope, conteúdo e transparência

```
import smtplib
corpo = ('From: aula@lab.test\r\nTo: aceito@lab.test\r\nSubject: Aula\r\n\r\n'
         '.linha iniciada por ponto\r\n..duas\r\n')
with smtplib.SMTP('127.0.0.1', 1025, timeout=5) as s:
    s.set_debuglevel(1)            # mostra comandos e respostas; não use com senhas reais
    s.ehlo('cliente.lab.test')
    print(s.mail('aula@lab.test'))
    print(s.rcpt('aceito@lab.test'))
    print(s.data(corpo))
```

Perguntas para responder com a saída: qual resposta pertence ao remetente e qual ao destinatário? Em que momento aparece 354? Qual o resultado após o conteúdo e qual ID foi devolvido? No debug, as linhas aparecem como `..linha` e `...duas`: o cliente acrescentou um ponto. No arquivo gravado, voltaram a `.linha` e `..duas`: o servidor removeu um ponto, conforme a RFC.

Localize o ID devolvido na pasta (`LAB*.eml`) e em `eventos.jsonl`. Esse arquivo não é caixa postal: serve para comparar bytes e eventos da transação. `s.data()` devolve código e texto da resposta final sem lançar exceção quando o servidor recusa o conteúdo; isso está no código da biblioteca e é confirmado pelo teste do capítulo 29.

29 Laboratório: falhas, aceitação parcial, SIZE e greylisting

```
import smtplib, time
msg = 'Subject: Parcial\r\n\r\nAula local.\r\n'
with smtplib.SMTP('127.0.0.1', 1025, timeout=5) as s:
    print(s.sendmail('aula@lab.test', ['aceito@lab.test', 'desconhecido@lab.test'], msg))
    # {'desconhecido@lab.test': (550, b'5.1.1 Unknown recipient')}

with smtplib.SMTP('127.0.0.1', 1025, timeout=5) as s:
    s.ehlo('cliente.lab.test')
    print(s.mail('aula@lab.test', ['SIZE=2000000']))      # (552, ...) antes do conteúdo

for tentativa in range(3):
    with smtplib.SMTP('127.0.0.1', 1025, timeout=5) as s:
        s.ehlo('cliente.lab.test'); s.mail('nova@lab.test')
        print(tentativa, s.rcpt('greylist@lab.test'))      # 450, 450, depois 250
    time.sleep(2)
```

- **Aceitação parcial:** o dicionário inclui só o recusado; não repita automaticamente para ambos.
- **RCPT aceito, DATA recusado:** use `dados-temp` ou `dados-perm` sozinhos para ver a diferença de etapa: o 250 em RCPT não impede recusa posterior.
- **Greylisting:** a tripla é IP, remetente e destinatário. Mudar o remetente recomeça a espera; repetir cedo demais mantém o 450. A bateria de testes usa um remetente único por execução, e por isso pode ser repetida sem reiniciar o servidor.

30 Laboratório: STARTTLS com verificação real

```
import smtplib, ssl
ctx = ssl.create_default_context(cafile='cert.pem')    # confia só no certificado do laboratório
with smtplib.SMTP('127.0.0.1', 1025, timeout=5) as s:
    s.ehlo('cliente.lab.test')
    print('Antes:', s.has_extn('starttls'))
    s.starttls(context=ctx)
    s.ehlo('cliente.lab.test')                          # obrigatório depois do TLS [19]
    print('Depois:', s.has_extn('starttls'))
    print(s.sock.version())
```

Antes, STARTTLS está anunciado; depois, não reaparece, como exige a RFC 3207 [5]. O certificado é verificado de verdade: o nome conferido é `127.0.0.1`, coberto pelo SAN do certificado gerado. Os testes automatizados também provam o caso negativo: com um contexto padrão, sem o certificado local, a verificação falha com `SSLCertVerificationError`. Não resolva isso desativando a verificação.

O teste envia ainda MAIL logo após o STARTTLS, sem novo EHLO, e verifica a recusa 503 do simulador. Isso reforça que o estado é reiniciado; não é uma alegação de que todo servidor real responda com a mesma frase.

31 Laboratório: Postfix real contra o simulador NOVO

Para que logs, fila, DSN e falha de TLS deste manual fossem reais, um Postfix 3.8.6 foi configurado para usar o simulador como relay. **Faça isso apenas em uma VM ou contêiner dedicado**: a configuração substitui o `main.cf`.

```
# Laboratorio do Manual de Investigacao de Entrega de E-mails (Edicao 2)
compatibility_level = 3.6
myhostname = relay.lab.test
mydomain = lab.test
mydestination =
inet_interfaces = loopback-only
inet_protocols = ipv4
relayhost = [localhost]:1025
transport_maps = texthash:/etc/postfix/transport
smtp_host_lookup = native
smtp_tls_security_level = secure
smtp_tls_CAfile = /caminho/para/cert.pem
smtp_tls_policy_maps = texthash:/etc/postfix/tls_policy
smtp_tls_loglevel = 1
maillog_file = /var/log/postfix.log
# Tempos reduzidos apenas para o laboratorio (padroes: 300s, 300s, 4000s)
queue_run_delay = 10s
minimal_backoff_time = 10s
maximal_backoff_time = 20s
```

```
# /etc/postfix/transport  (tlserro.lab.test vai para o IP, com política própria)
tlserro.lab.test smtp:[127.0.0.1]:1025
# /etc/postfix/tls_policy (exige um nome que o certificado não cobre)
[127.0.0.1]:1025 secure match=nome-errado.lab.test
```

31.1 Passos executados EXECUTADO

01 Instalar o pacote `postfix` sem configuração automática; no `master.cf`, desativar o serviço `smtp inet` (o laboratório não recebe na porta 25) e desligar o chroot dos serviços.

02 Gerar o certificado e iniciar o simulador (capítulo 27); trocar o caminho do `smtp_tls_CAfile` pelo do seu `cert.pem`.

03 `postfix check` e `postfix start`.

04 `sh enviar_testes_postfix.sh` e acompanhar `/var/log/postfix.log`, `postqueue -p` e `postqueue -j`.

31.2 Resultado das cinco mensagens

Destinatário	Resultado no Postfix	Onde está no manual
aceito@lab.test	`status=sent`, TLS verificado	Capítulo 19
desconhecido@lab.test	`status=bounced`, DSN entregue ao remetente	Capítulos 17 e 19
greylist@lab.test	`deferred` 450 4.7.1, aceita na nova tentativa	Capítulos 8, 14 e 15
temporario@lab.test	Permanece `deferred`	Capítulo 13
aceito@tlserro.lab.test	`deferred`, `dsn=4.7.5`, certificado não verificado	Capítulos 13 e 26

Os tempos de nova tentativa foram reduzidos para 10 s apenas para o laboratório; os padrões estão no capítulo 14. Ao terminar: `postfix stop` e restaure o `main.cf` e o `master.cf` originais.

32 Resultados verificados e o que eles não provam

`testar_smtp.py` executado contra o simulador nesta edição (Python 3.13.18, OpenSSL 3.0.13), duas vezes seguidas, sem reiniciar o servidor **EXECUTADO**:

```
OK EHLO anuncia STARTTLS e SIZE
OK RCPT antes de MAIL -> 503
OK MAIL aceito -> 250
OK RCPT temporario -> 450
OK RCPT inexistente -> 550
OK RCPT aceito -> 250
OK Resultado apos DATA -> 250 com ID
OK Transparencia do ponto preserva o conteudo
OK RSET descarta o envelope (DATA -> 503)
OK RCPT aceito, DATA recusado -> 451
OK RCPT aceito, DATA recusado -> 554
OK Aceitacao parcial: so o recusado volta no dicionario
OK Todos recusados -> SMTPRecipientsRefused
OK SIZE acima do limite -> 552 em MAIL
OK Greylisting: primeira tentativa -> 450 4.7.1
OK Greylisting: repeticao imediata -> ainda 450
OK Greylisting: apos o atraso minimo -> 250
OK STARTTLS com certificado validado -> 220
OK MAIL sem novo EHLO apos TLS -> 503
OK STARTTLS nao e anunciado de novo
OK Mensagem aceita dentro do TLS
OK Sem confiar no certificado local -> verificacao falha
22 verificacoes aprovadas

(segunda execucao, sem reiniciar o servidor)
22 verificacoes aprovadas
```

32.1 Limites da validação

A bateria comprova os comportamentos programados do simulador e o diálogo do cliente, incluindo TLS verificado. O capítulo 31 comprova o comportamento de um Postfix real diante desses cenários. Nada disso valida bloqueios de rede, entrega externa, DNS público, políticas de provedores ou a configuração do seu servidor de produção: a validação de uma correção real exige repetir o fluxo autorizado no ambiente afetado.

33 Código do simulador

servidor_smtp.py — reescrito nesta edição.

```
"""Simulador SMTP local do Manual de Investigacao de Entrega de E-mails (Edicao 2).

Escuta somente em 127.0.0.1 (porta 1025 por padrao). Nao retransmite, nao entrega
em caixa postal e nao e um MTA: grava o conteudo aceito em LAB*.eml e registra
eventos em eventos.jsonl, para comparacao com o que o cliente enviou.

Variaveis de ambiente:
  SMTP_LAB_DIR    pasta de trabalho (cert.pem, key.pem, mensagens e eventos)
  SMTP_LAB_PORT   porta TCP (padrao 1025)
  SMTP_LAB_GREY   atraso minimo do greylisting, em segundos (padrao 3)
"""
import json
import os
import re
import socketserver
import ssl
import threading
import time
import uuid
from datetime import datetime, timezone
from pathlib import Path

BASE = Path(os.environ.get('SMTP_LAB_DIR', Path(__file__).parent))
PORT = int(os.environ.get('SMTP_LAB_PORT', '1025'))
GREY_DELAY = float(os.environ.get('SMTP_LAB_GREY', '3'))
MAX_SIZE = 1048576                      # anunciado em EHLO como SIZE (RFC 1870)
LOCK = threading.Lock()
GREYLIST = {}                           # (ip, remetente, destinatario) -> primeira tentativa

ACEITOS = {'aceito@lab.test', 'dados-temp@lab.test', 'dados-perm@lab.test'}


def contexto_tls():
    ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    ctx.load_cert_chain(BASE / 'cert.pem', BASE / 'key.pem')
    return ctx


def evento(tipo, **campos):
    linha = {'time': datetime.now(timezone.utc).isoformat(), 'event': tipo, **campos}
    with LOCK, (BASE / 'eventos.jsonl').open('a', encoding='utf-8') as saida:
        saida.write(json.dumps(linha) + '\n')


def greylisting(ip, remetente, destinatario):
    """Greylisting por tripla (RFC 6647): recusa temporaria ate passar o atraso minimo."""
    chave = (ip, remetente.lower(), destinatario)
    agora = time.monotonic()
    with LOCK:
        primeira = GREYLIST.setdefault(chave, agora)
    return agora - primeira >= GREY_DELAY


class Handler(socketserver.BaseRequestHandler):
    def handle(self):
        sock = self.request
        sock.settimeout(30)
        leitor = sock.makefile('rb')
        ip = self.client_address[0]
        ehlo, tls = False, False
```

```
        remetente, destinatarios = None, []

        def enviar(texto):
            sock.sendall(texto.encode('ascii') + b'\r\n')

        enviar('220 localhost ESMTP laboratorio')
        try:
            while True:
                linha = leitor.readline(1001)
                if not linha:
                    return
                if not linha.endswith(b'\n'):
                    enviar('500 5.5.2 Line too long')
                    return
                texto = linha.decode('ascii', 'replace').rstrip('\r\n')
                verbo, _, arg = texto.partition(' ')
                verbo = verbo.upper()

                if verbo in ('EHLO', 'HELO'):
                    ehlo = True
                    remetente, destinatarios = None, []
                    if verbo == 'HELO':
                        enviar('250 localhost')
                        continue
                    enviar('250-localhost')
                    enviar(f'250-SIZE {MAX_SIZE}')
                    if not tls:
                        enviar('250-STARTTLS')         # RFC 3207: nao anunciar apos TLS
                    enviar('250 ENHANCEDSTATUSCODES')

                elif verbo == 'STARTTLS':
                    if not ehlo or tls or remetente is not None:
                        enviar('503 5.5.1 Invalid state')
                        continue
                    enviar('220 2.0.0 Ready for TLS')
                    # Descarta qualquer texto em claro ja lido apos o comando STARTTLS.
                    leitor.close()
                    sock = contexto_tls().wrap_socket(sock, server_side=True)
                    leitor = sock.makefile('rb')
                    tls, ehlo = True, False             # RFC 3207: estado reiniciado
                    remetente, destinatarios = None, []
                    evento('tls', client=ip, version=sock.version())

                elif verbo == 'MAIL':
                    if not ehlo:
                        enviar('503 5.5.1 EHLO required')
                        continue
                    m = re.fullmatch(r'FROM:<([^>]*)>(.*)', arg, re.I)
                    if not m:
                        enviar('501 5.5.2 Syntax error')
                        continue
                    tamanho = re.search(r'\bSIZE=(\d+)', m[2], re.I)
                    if tamanho and int(tamanho[1]) > MAX_SIZE:
                        enviar('552 5.3.4 Message size exceeds fixed limit')
                        evento('mail_rejected', sender=m[1], size=int(tamanho[1]))
                        continue
                    remetente, destinatarios = m[1], []
                    enviar('250 2.1.0 Sender accepted')

                elif verbo == 'RCPT':
                    if remetente is None:
                        enviar('503 5.5.1 MAIL required')
                        continue
                    m = re.fullmatch(r'TO:<([^>]+)>.*', arg, re.I)
                    endereco = m[1].lower() if m else ''
```

```
                if endereco == 'temporario@lab.test':
                    resposta = '450 4.2.0 Temporary recipient failure'
                elif endereco == 'greylist@lab.test':
                    if greylisting(ip, remetente, endereco):
                        resposta = '250 2.1.5 Recipient accepted'
                    else:
                        resposta = '450 4.7.1 Greylisted, try again later'
                elif endereco in ACEITOS:
                    resposta = '250 2.1.5 Recipient accepted'
                else:
                    resposta = '550 5.1.1 Unknown recipient'
                if resposta.startswith('250'):
                    destinatarios.append(endereco)
                enviar(resposta)
                evento('rcpt', client=ip, sender=remetente, recipient=endereco,
                       code=int(resposta[:3]))

            elif verbo == 'DATA':
                if not destinatarios:
                    enviar('503 5.5.1 RCPT required')
                    continue
                enviar('354 End data with <CRLF>.<CRLF>')
                corpo = bytearray()
                while True:
                    dados = leitor.readline(65537)
                    if not dados:
                        return
                    if dados == b'.\r\n':
                        break
                    # RFC 5321, 4.5.2: linha iniciada por ponto e com outros
                    # caracteres perde o primeiro ponto.
                    if dados.startswith(b'.'):
                        dados = dados[1:]
                    corpo.extend(dados)
                    if len(corpo) > MAX_SIZE:
                        enviar('552 5.3.4 Message size exceeds fixed limit')
                        return
                if 'dados-temp@lab.test' in destinatarios:
                    enviar('451 4.3.0 Temporary processing failure')
                    evento('data_rejected', client=ip, code=451)
                elif 'dados-perm@lab.test' in destinatarios:
                    enviar('554 5.7.1 Message rejected by lab policy')
                    evento('data_rejected', client=ip, code=554)
                else:
                    qid = 'LAB' + uuid.uuid4().hex[:12].upper()
                    (BASE / f'{qid}.eml').write_bytes(bytes(corpo))
                    evento('accepted', client=ip, queue_id=qid, sender=remetente,
                           recipients=destinatarios, tls=tls, size=len(corpo))
                    enviar(f'250 2.0.0 Stored locally as {qid}')
                remetente, destinatarios = None, []

            elif verbo == 'RSET':
                remetente, destinatarios = None, []
                enviar('250 2.0.0 Reset')
            elif verbo == 'NOOP':
                enviar('250 2.0.0 OK')
            elif verbo == 'QUIT':
                enviar('221 2.0.0 Bye')
                return
            else:
                enviar('502 5.5.1 Command not implemented')
        except (OSError, ssl.SSLError) as exc:
            evento('connection_error', client=ip, error=type(exc).__name__)
        finally:
            leitor.close()
```

```
            sock.close()


class Servidor(socketserver.ThreadingTCPServer):
    allow_reuse_address = True
    daemon_threads = True


if __name__ == '__main__':
    contexto_tls()  # falha cedo se cert.pem/key.pem nao existirem
    with Servidor(('127.0.0.1', PORT), Handler) as servidor:
        print(f'SMTP de laboratorio em 127.0.0.1:{PORT}; Ctrl+C encerra.', flush=True)
        try:
            servidor.serve_forever()
        except KeyboardInterrupt:
            pass
```

preparar_tls.py

```
"""Gera um certificado autoassinado de laboratorio (localhost e 127.0.0.1), valido por 7 dias.

Nao instale este certificado na confianca global do sistema e nao reutilize a chave.
"""
import os
import subprocess
from pathlib import Path

base = Path(os.environ.get('SMTP_LAB_DIR', Path(__file__).parent))
base.mkdir(parents=True, exist_ok=True)
chave, cert = base / 'key.pem', base / 'cert.pem'
if chave.exists() or cert.exists():
    raise SystemExit('cert.pem ou key.pem ja existem. Preserve-os ou use uma pasta nova.')

versao = subprocess.run(['openssl', 'version'], capture_output=True, text=True, check=True).stdout
# OpenSSL 3.0 marcou -nodes como obsoleto em favor de -noenc.
sem_senha = '-noenc' if versao.split()[1].startswith(('3', '4')) else '-nodes'

subprocess.run(['openssl', 'req', '-x509', '-newkey', 'rsa:2048', sem_senha,
                '-keyout', str(chave), '-out', str(cert), '-days', '7',
                '-subj', '/CN=localhost',
                '-addext', 'subjectAltName=DNS:localhost,IP:127.0.0.1'],
               check=True, capture_output=True)
chave.chmod(0o600)
print(f'{versao.strip()}: certificado de laboratorio criado em {base} (7 dias).')
```

34 Código dos testes

testar_smtp.py — 22 verificações.

```
"""Testes do laboratorio SMTP (Edicao 2). Rode com servidor_smtp.py ativo.

Pode ser repetido sem reiniciar o servidor: cada execucao usa um remetente unico.
"""
import os
import smtplib
import ssl
import time
import uuid
from pathlib import Path

BASE = Path(os.environ.get('SMTP_LAB_DIR', Path(__file__).parent))
PORTA = int(os.environ.get('SMTP_LAB_PORT', '1025'))
GREY = float(os.environ.get('SMTP_LAB_GREY', '3'))
REMETENTE = f'aula-{uuid.uuid4().hex[:8]}@lab.test'
CORPO = ('From: aula@lab.test\r\nTo: aceito@lab.test\r\n'
         'Message-ID: <ensaio@lab.test>\r\nSubject: Aula local\r\n\r\n'
         '.linha iniciada por ponto\r\n..duas\r\nfim\r\n')
total = 0


def ok(nome, condicao):
    global total
    assert condicao, nome
    total += 1
    print('OK', nome)


def conectar():
    return smtplib.SMTP('127.0.0.1', PORTA, timeout=5)


# 1. Sessao, envelope e conteudo
with conectar() as s:
    codigo, _ = s.ehlo('cliente.lab.test')
    ok('EHLO anuncia STARTTLS e SIZE',
       codigo == 250 and s.has_extn('starttls') and s.has_extn('size'))
    ok('RCPT antes de MAIL -> 503', s.rcpt('aceito@lab.test')[0] == 503)
    ok('MAIL aceito -> 250', s.mail(REMETENTE)[0] == 250)
    ok('RCPT temporario -> 450', s.rcpt('temporario@lab.test')[0] == 450)
    ok('RCPT inexistente -> 550', s.rcpt('desconhecido@lab.test')[0] == 550)
    ok('RCPT aceito -> 250', s.rcpt('aceito@lab.test')[0] == 250)
    codigo, texto = s.data(CORPO)
    ok('Resultado apos DATA -> 250 com ID', codigo == 250 and b'LAB' in texto)
    gravado = (BASE / (texto.decode().split()[-1] + '.eml')).read_bytes()
    ok('Transparencia do ponto preserva o conteudo',
       b'\r\n.linha iniciada por ponto\r\n..duas\r\n' in gravado)
    s.mail(REMETENTE)
    s.rcpt('aceito@lab.test')
    s.rset()
    ok('RSET descarta o envelope (DATA -> 503)', s.docmd('DATA')[0] == 503)

# 2. Recusa depois do conteudo: data() devolve o codigo, sem excecao
for destinatario, esperado in (('dados-temp@lab.test', 451), ('dados-perm@lab.test', 554)):
    with conectar() as s:
        s.ehlo('cliente.lab.test')
        s.mail(REMETENTE)
        s.rcpt(destinatario)
        ok(f'RCPT aceito, DATA recusado -> {esperado}', s.data(CORPO)[0] == esperado)
```

```
# 3. sendmail(): aceitacao parcial e recusa total
with conectar() as s:
    recusados = s.sendmail(REMETENTE, ['aceito@lab.test', 'desconhecido@lab.test'], CORPO)
    ok('Aceitacao parcial: so o recusado volta no dicionario',
       set(recusados) == {'desconhecido@lab.test'})
with conectar() as s:
    try:
        s.sendmail(REMETENTE, ['desconhecido@lab.test'], CORPO)
    except smtplib.SMTPRecipientsRefused as exc:
        ok('Todos recusados -> SMTPRecipientsRefused', 'desconhecido@lab.test' in exc.recipients)
    else:
        raise AssertionError('SMTPRecipientsRefused esperado')

# 4. SIZE declarado acima do limite anunciado
with conectar() as s:
    s.ehlo('cliente.lab.test')
    ok('SIZE acima do limite -> 552 em MAIL', s.mail(REMETENTE, ['SIZE=2000000'])[0] == 552)

# 5. Greylisting por tripla (IP, remetente, destinatario)
def rcpt_grey():
    with conectar() as s:
        s.ehlo('cliente.lab.test')
        s.mail(REMETENTE)
        return s.rcpt('greylist@lab.test')

codigo, texto = rcpt_grey()
ok('Greylisting: primeira tentativa -> 450 4.7.1', codigo == 450 and b'4.7.1' in texto)
ok('Greylisting: repeticao imediata -> ainda 450', rcpt_grey()[0] == 450)
time.sleep(GREY + 0.5)
ok('Greylisting: apos o atraso minimo -> 250', rcpt_grey()[0] == 250)

# 6. STARTTLS com verificacao real do certificado de laboratorio
contexto = ssl.create_default_context(cafile=str(BASE / 'cert.pem'))
with conectar() as s:
    s.ehlo('cliente.lab.test')
    ok('STARTTLS com certificado validado -> 220', s.starttls(context=contexto)[0] == 220)
    ok('MAIL sem novo EHLO apos TLS -> 503',
       s.docmd('MAIL', f'FROM:<{REMETENTE}>')[0] == 503)
    s.ehlo('cliente.lab.test')
    ok('STARTTLS nao e anunciado de novo', not s.has_extn('starttls'))
    ok('Mensagem aceita dentro do TLS', s.sendmail(REMETENTE, ['aceito@lab.test'], CORPO) == {})
with conectar() as s:
    s.ehlo('cliente.lab.test')
    try:
        s.starttls(context=ssl.create_default_context())   # sem o certificado local
    except ssl.SSLCertVerificationError:
        ok('Sem confiar no certificado local -> verificacao falha', True)
    else:
        raise AssertionError('falha de verificacao esperada')

print(f'{total} verificacoes aprovadas')
```

enviar_testes_postfix.sh

```
#!/bin/sh
# enviar_testes_postfix.sh - submete cinco mensagens de laboratorio ao Postfix local.
# Use SOMENTE em VM ou conteiner dedicado, com o main.cf.lab aplicado (capitulo 31).
# Todas as mensagens vao para o simulador em 127.0.0.1:1025; nada sai da maquina.
set -eu
mk() {
    printf 'From: ERP <aceito@lab.test>\nTo: %s\nSubject: %s\nMessage-ID: <%s@lab.test>\nDate:
%s\n\nMensagem de laboratorio.\n' \
        "$1" "$2" "$3" "$(date -R)"
}
```

```
for r in aceito desconhecido greylist temporario; do
    mk "$r@lab.test" "Teste $r" "manual-$r" | /usr/sbin/sendmail -f aceito@lab.test "$r@lab.test"
done
mk aceito@tlserro.lab.test "Teste TLS" manual-tls | /usr/sbin/sendmail -f aceito@lab.test
aceito@tlserro.lab.test
echo "5 mensagens submetidas. Acompanhe: tail -f /var/log/postfix.log ; postqueue -p"
```

35 Relatório e linha do tempo

O relatório deve permitir reconstruir a transação e localizar a responsabilidade atual. Evite "SMTP deu erro" sem código, etapa e destino; inclua o resultado por destinatário e a última transferência confirmada.

Campo	Conteúdo necessário
Identificação	ID do incidente, analista, data e fuso
Fluxo	Aplicação, relay, servidor remoto e entrega local
Envelope	MAIL FROM e RCPT TO relevantes
Correlação	Queue-ID por hop e Message-ID
DNS / rota	MX ou relay configurado, IP, família e porta usados
Sessão	Última etapa concluída, código e resposta completa
TLS / AUTH	Política exigida, resultado (Verified/Untrusted) e nome verificado; sem segredos
Fila	Estado, idade, tentativas e motivo corrente
Causa	Hipótese confirmada e evidência que a sustenta
Mudança	Correção, responsável, horário e rollback
Aceite	Teste no caminho original e regressão
Limitações	Componentes sem acesso e fatos não observados

35.1 Linha do tempo e estado conhecido

Registre submissão, aceitação local, cada tentativa relevante, resposta remota e eventual entrega ou devolução. Converta fusos para comparar e preserve o original. Se há sucesso para o relay mas não há rastreamento da caixa, registre essa fronteira; se houve timeout após o conteúdo, registre a incerteza e o risco de duplicidade; se a primeira consulta DNS foi feita depois do incidente, ela descreve outro instante. Ao compartilhar, reduza dados pessoais e conteúdo ao necessário.

36 Mudanças seguras e critérios de aceite

Antes de intervir, identifique quem mantém a mensagem pendente, salve a configuração relevante, o estado da fila e os eventos da tentativa, e não modifique várias camadas ao mesmo tempo.

Problema confirmado	Correção com foco	Validação
MX/host errado	Publicação ou rota correta	DNS e conexão a partir do MTA afetado
Porta bloqueada	Regra de rede prevista	Banner e transação permitida
Destinatário inválido	Cadastro/alias confirmado	Aceitação do endereço correto
TLS inválido	Certificado, cadeia e nomes corretos	`posttls-finger` e entrega com a política original
IPv6 sem PTR	PTR e DNS direto do IP de saída	Entrega pela mesma família de IP
Recusa de relay	Autorização específica	Fluxo legítimo sem ampliar acesso
Fila crescente	Causa compartilhada tratada	Idade e volume convergindo

Correções que escondem a causa: desativar TLS, liberar relay amplo, ignorar erros permanentes, encurtar intervalos de tentativa para "vencer" greylisting ou reenviar a fila inteira. **Rollback com estado:** mensagens já aceitas por outro servidor não são "desenviadas"; considere também caches de DNS, certificados ativos, políticas de transporte e duplicidades. **Aceite:** um destinatário válido, um inválido e um temporário controlado; TLS, tamanho ou IPv6 se fizeram parte da causa; e a fila observada depois da mudança. Encerre explicando por que a mudança resolveu o mecanismo e como a recorrência será detectada (idade de fila, taxa de adiamento, falhas por destino).

A Exercícios de investigação

Para cada cenário, indique a última etapa confirmada, quem mantém a responsabilidade e qual evidência falta.

1 — O 250 prematuro

O log da aplicação mostra 250 para RCPT e 451 depois de DATA. O desenvolvedor afirma que o destinatário recebeu porque "houve 250". Qual resposta decide a aceitação?

2 — Dois destinatários

Ana recebe 250 em RCPT; Bruno, 550. O conteúdo é aceito. A aplicação reenvia para os dois, tratando o resultado como falha total. Que risco foi criado?

3 — Timeout depois do conteúdo

O cliente enviou todos os bytes e perdeu a conexão antes de ler o resultado final. O receptor registra armazenamento e sucesso. Como classificar a situação para o cliente?

4 — DNS versus rede

MX e A respondem, mas a conexão TCP/25 do relay expira para vários domínios. Um notebook conecta. Qual ponto de teste deve orientar o diagnóstico?

5 — Cadeia TLS

Um `s_client` sem verificação de nome negocia TLS. O MTA exige identidade verificada e adia a mensagem. Que teste acrescentar antes de culpar o MTA?

6 — Entregue ao próximo hop

O Postfix registra `status=sent` com ID remoto. O usuário não encontra a mensagem. Qual equipe e quais identificadores continuam o rastreamento?

7 — Nova tentativa

Um RCPT recebe 450. A aplicação encerra sem guardar a mensagem. Pode-se presumir retry automático só porque o código era temporário?

8 — Quanto tempo na fila?

Um parceiro está fora do ar há três dias e o seu Postfix usa os valores padrão. O que acontece com as mensagens adiadas, e quando?

9 — Greylisting que nunca libera NOVO

Mensagens de um sistema de marketing para um cliente são adiadas com `450 4.7.1 Greylisted` a cada tentativa, por horas. Outros remetentes chegam normalmente. Qual a hipótese mais provável?

10 — Só o IPv6 falha NOVO

Mensagens ao Gmail falham com `550-5.7.1 ... IPv6 sending guidelines regarding PTR records` quando o relay sai por IPv6; por IPv4 passam. O que verificar e o que **não** mexer?

11 — Verified ou Untrusted NOVO

Com a política `secure` e o CAfile correto, o `posttls-finger` mostra "Untrusted TLS connection" e, antes, "num=62:hostname mismatch". O código de saída foi 0. O destino está bom para essa política?

12 — Laboratório NOVO

Na Edição 1, o teste de nova tentativa falhava ao repetir a bateria sem reiniciar o servidor. Por que a Edição 2 não tem esse problema, e por que isso também imita melhor o greylisting real?

B Gabarito comentado

1

O 451 após DATA: a transferência não terminou com aceitação. O 250 anterior tinha escopo de RCPT. O remetente responsável precisa tratar o trabalho pendente.

2

Ana pode receber cópia duplicada. Guarde o estado por destinatário e trate só o recusado, conforme a causa.

3

Resultado incerto para o cliente. Os logs do receptor esclarecem aquele hop, mas uma nova tentativa automática pode duplicar. Correlacione os dois lados antes de recomendar reenvio.

4

O relay: rede, NAT, política de saída e serviço a partir dele. O notebook usa outro caminho; DNS funcional não prova conectividade TCP, e falha de conexão não se corrige no SPF.

5

`posttls-finger` com o mesmo nível de segurança e o mesmo nome exigido, a partir do MTA; ou `openssl s_client` com `-verify_hostname` e `-verify_return_error`. SNI não é verificação de nome.

6

A equipe que controla o servidor que aceitou, com ID remoto, horário, envelope e Message-ID. Sem essa visibilidade, conclua apenas a transferência comprovada.

7

Não. Código temporário não cria armazenamento nem agendamento. A aplicação precisa implementar o tratamento ou submeter a um MTA com fila.

8

Com os padrões, o Postfix tenta de novo depois de cerca de 5 minutos, com intervalos crescentes até 4000 s (pouco mais de uma hora). Ao completar 5 dias na fila (`maximal_queue_lifetime`), a falha temporária vira devolução, com DSN ao remetente. Em três dias, as mensagens continuam na fila adiada.

9

O sistema de marketing envia cada tentativa por um IP diferente de um pool. Como o greylisting lembra a tripla IP + remetente + destinatário, a tripla nunca se repete e a espera recomeça. Confirme os IPs de saída no log; soluções: fixar a saída para esse fluxo ou o receptor incluir o pool na lista de exceção.

10

Verifique o PTR do IPv6 de saída e o DNS direto correspondente, e a autenticação, como pede o próprio texto da recusa. Não mexa no MX do destino nem acrescente IPs ao SPF sem evidência: o problema está na identidade do IP de origem naquela família.

11

Não. "Untrusted" com `hostname mismatch` significa que o certificado não cobre o nome exigido; um MTA com `secure` adiaria a entrega, como no capítulo 26 (`dsn=4.7.5`). O código 0 do programa não indica sucesso de verificação: leia o texto.

12

A Edição 1 usava um contador global em memória. A Edição 2 implementa greylisting por tripla com atraso mínimo, e os testes usam remetente único por execução, então cada execução começa uma tripla nova. É exatamente como o greylisting real decide.

C Referência rápida por etapa

Última evidência	Próximo foco	Capítulo	Evite inferir
MX encontrado	A/AAAA, rota efetiva, conexão	3, 4	Serviço SMTP disponível
TCP aberto	Banner e diálogo	4, 5	Mensagem aceita
EHLO concluído	Capacidades e política	9	AUTH/TLS corretos
TLS negociado	Verified ou Untrusted; nome e política	10, 26	"TLS está bom"
AUTH aprovado	MAIL, RCPT e restrições	11	Relay irrestrito
MAIL 552 5.3.4	SIZE declarado e limites	9	Problema de rede
RCPT 250	Resultado após o conteúdo	5, 12	Caixa recebeu
RCPT 4xx 4.7.1	Greylisting, taxa, política; nova tentativa	15, 16	Falha permanente
RCPT 5xx	Endereço/política e correção	7, 8, 23	Retry infinito resolve
550 5.7.1 IPv6 / PTR	PTR e DNS do IP de saída	16	Erro de MX ou SPF
DATA 250 final	Próximo hop / entrega local	12, 25	Leitura do usuário
Timeout final	Logs dos dois lados	14	Reenvio sem risco
`status=deferred`	Motivo, idade, tentativas	13, 14	Mensagem perdida
`dsn=4.7.5`	Verificação de certificado no MTA	26	Destino fora do ar
`status=bounced`	DSN e Diagnostic-Code	17	Remetente notificado pelo destino
`status=sent`	Destino e resposta remota	12, 19	Caixa de entrada garantida

Comandos essenciais

Tarefa	Comando	Observação
Fila	`postqueue -p` · `postqueue -j`	Somente leitura; `-j` desde o Postfix 3.1
Concentração	`qshape deferred`	**RPM** pacote `postfix-perl-scripts`
Item da fila	`postcat -q ID`	Expõe conteúdo
Valores efetivos e padrão	`postconf PARAM` · `postconf -d PARAM`	
TLS como o Postfix	`posttls-finger -c -l secure -F ca.pem "[mx]:25"`	Leia Verified/Untrusted
TLS no terminal	`openssl s_client -starttls smtp ... -verify_hostname N -verify_return_error`	Sem as duas opções, "passa" sempre
Log	`grep ID /var/log/mail.log` (DEB) · `/var/log/maillog` (RPM)	Ou journal / `maillog_file`

Perguntas para concluir

01 Qual mensagem e qual destinatário?

02 Qual rota, IP e família foram usados?

03 Qual operação falhou, e quem respondeu?

04 Houve aceitação final do conteúdo?

05 Quem mantém o próximo passo, e a evidência confirma só transferência ou também entrega local?

D Fontes oficiais

Documentos da IETF/RFC Editor, dos projetos e dos provedores citados. Estudos de caso, tabelas de investigação e exercícios são construções didáticas. Consulte atualizações, erratas e a versão instalada antes de aplicar parâmetros.

[1] RFC 5321 — SMTP
https://www.rfc-editor.org/rfc/rfc5321.html

[2] RFC 5322 — Internet Message Format
https://www.rfc-editor.org/rfc/rfc5322.html

[3] RFC 6409 — Message Submission
https://www.rfc-editor.org/rfc/rfc6409.html

[4] RFC 8314 — TLS for Submission and Access
https://www.rfc-editor.org/rfc/rfc8314.html

[5] RFC 3207 — STARTTLS
https://www.rfc-editor.org/rfc/rfc3207.html

[6] RFC 4954 — SMTP AUTH
https://www.rfc-editor.org/rfc/rfc4954.html

[7] RFC 3463 — Enhanced Status Codes
https://www.rfc-editor.org/rfc/rfc3463.html

[8] RFC 2034 — Enhanced Error Codes
https://www.rfc-editor.org/rfc/rfc2034.html

[9] RFC 3461 — DSN Extension
https://www.rfc-editor.org/rfc/rfc3461.html

[10] RFC 3464 — DSN Format
https://www.rfc-editor.org/rfc/rfc3464.html

[11] RFC 7505 — Null MX
https://www.rfc-editor.org/rfc/rfc7505.html

[12] Postfix — postqueue(1)
https://www.postfix.org/postqueue.1.html

[13] Postfix — postcat(1)
https://www.postfix.org/postcat.1.html

[14] Postfix — Bottleneck Analysis (qshape)
https://www.postfix.org/QSHAPE_README.html

[15] Postfix — postconf(5)
https://www.postfix.org/postconf.5.html
delays=a/b/c/d, tempos de fila, smtp_tls_loglevel, maillog_file.

[16] Postfix — Debugging Howto
https://www.postfix.org/DEBUG_README.html

[17] Postfix — TLS Support
https://www.postfix.org/TLS_README.html

[18] OpenSSL 3.0 — s_client
https://docs.openssl.org/3.0/man1/openssl-s_client/

[19] Python 3.13 — smtplib
https://docs.python.org/3.13/library/smtplib.html

[20] RFC 1870 — SIZE
https://www.rfc-editor.org/rfc/rfc1870.html

[21] RFC 2920 — PIPELINING
https://www.rfc-editor.org/rfc/rfc2920.html

[22] RFC 3030 — CHUNKING
https://www.rfc-editor.org/rfc/rfc3030.html

[23] RFC 6531 — SMTPUTF8
https://www.rfc-editor.org/rfc/rfc6531.html

[24] RFC 8601 — Authentication-Results
https://www.rfc-editor.org/rfc/rfc8601.html

[25] RFC 2033 — LMTP
https://www.rfc-editor.org/rfc/rfc2033.html

[26] RFC 8461 — MTA-STS
https://www.rfc-editor.org/rfc/rfc8461.html

[27] RFC 7672 — DANE for SMTP
https://www.rfc-editor.org/rfc/rfc7672.html

[28] RFC 8460 — TLSRPT
https://www.rfc-editor.org/rfc/rfc8460.html

[29] Postfix — Architecture Overview
https://www.postfix.org/OVERVIEW.html

[30] RFC 6152 — 8BITMIME
https://www.rfc-editor.org/rfc/rfc6152.html

[31] Postfix — SMTP Relay and Access Control
https://www.postfix.org/SMTPD_ACCESS_README.html

[32] Postfix — posttls-finger(1)
https://www.postfix.org/posttls-finger.1.html

[33] RFC 6647 — Email Greylisting
https://www.rfc-editor.org/rfc/rfc6647.html

[34] Google — Email sender guidelines (Gmail)
https://support.google.com/mail/answer/81126

[35] Oracle Linux 9 AppStream — postfix-perl-scripts (qshape, pflogsumm)
https://yum.oracle.com/repowatch/oraclelinux9.x86_64/ol9_appstream/

[36] OpenSSL 3.0 — openssl-req (-noenc)
https://docs.openssl.org/3.0/man1/openssl-req/

E Registro de edições e verificação

Edição 2 — 29 de outubro de 2026

- Novos capítulos 15 (greylisting), 16 (exigências dos grandes receptores) e 31 (Postfix real contra o simulador).
- Capítulo 10 com execução real de `openssl s_client` e `posttls-finger` e o `smtp_tls_loglevel`; capítulo 14 com os valores padrão de tentativa; caso 26 reproduzido de verdade.
- Logs, fila, DSN e falha de TLS dos capítulos 8, 13, 14, 17, 19 e 26 vêm de execução real do Postfix 3.8.6.
- Laboratório reescrito: transparência do ponto conforme a RFC 5321 (a Edição 1 só removia o ponto de linhas iniciadas por dois pontos); greylisting por tripla no lugar do contador global, com testes repetíveis sem reiniciar; SIZE; recusa total no `sendmail()`; caso negativo de verificação TLS; `-noenc` e permissão 600 na chave. De 18 para 22 testes.
- Correções: `grep` no lugar de `rg`; `qshape` no RHEL; local do log por família; rotas de leitura por capítulo; exercícios, gabarito, referência e fontes como apêndices; tabela de MX reescrita; códigos 4.7.1 e 4.7.5 acrescentados.
- Exercícios 8 a 12 novos ou reescritos, com gabarito.

Edição 1 — 29 de outubro de 2026

- Publicação original: fundamentos, sessão SMTP, filas, cinco casos e simulador com 18 testes.

O que foi verificado

- `testar_smtp.py`: 22 de 22 aprovados, executado duas vezes seguidas sem reiniciar o servidor (Ubuntu 24.04.5, Python 3.13.18, OpenSSL 3.0.13).
- Postfix 3.8.6 do Ubuntu 24.04 configurado conforme o capítulo 31: cinco mensagens com os resultados descritos; `enviar_testes_postfix.sh` executado de novo para validar o script.
- Parâmetros e textos do Postfix conferidos na fonte da documentação (`postconf.proto`) e nas páginas de manual do pacote; regras da RFC 3207, da RFC 6647 e do Gmail conferidas nas páginas oficiais.
- Transcrições de produção, nomes de servidores reais e os casos 22 a 25 são construções didáticas. Nenhuma mensagem foi enviada para fora da máquina de laboratório.